
Positive Distribution Shift as a Framework for Understanding Tractable Learning

Marko Medvedev¹ Idan Attias^{2,3} Elisabetta Cornacchia⁴ Theodor Misiakiewicz⁵ Gal Vardi⁶ Nathan Srebro²

Abstract

We study a setting where the goal is to learn a target function $f(x)$ with respect to a target distribution $D(x)$, but training is done on i.i.d. samples from a different training distribution $D'(x)$, labeled by the true target $f(x)$. Such a distribution shift (here in the form of covariate shift) is usually viewed negatively, as hurting or making learning harder, and the traditional distribution shift literature is mostly concerned with limiting or avoiding this negative effect. In contrast, we argue that with a well-chosen $D'(x)$, the shift can be positive and make learning easier – a perspective called *Positive Distribution Shift (PDS)*. Such a perspective is central to contemporary machine learning, where much of the innovation is in finding good training distributions $D'(x)$, rather than changing the training algorithm. We further argue that the benefit is often computational rather than statistical, and that PDS allows computationally hard problems to become tractable even using standard gradient-based training. We formalize different variants of PDS, show how certain hard classes are easily learnable under PDS, and make connections with membership query learning.

1. Introduction

We consider a setting where our goal is to obtain good performance on some *target distribution* $\mathcal{D}$, but instead train on i.i.d. data from a different *training distribution* $\mathcal{D}'$. This type of distribution shift scenario is usually seen as less preferable than training on data from the target $\mathcal{D}$ itself. For

example, we may want accurate pedestrian detection on images taken from cars in New York, but only have training data collected in California. Or we may wish to classify sentiment in scientific papers, yet train on Reddit posts. In practice, we rely on $\mathcal{D}'$ as a proxy because $\mathcal{D}$ is inaccessible, too costly to collect, or only available in very small amounts, while data from a different, but related, $\mathcal{D}'$ is easier to obtain. Indeed, most of the distribution shift literature is concerned with ensuring that training on $\mathcal{D}'$ is not (much) worse than training on $\mathcal{D}$.

The phenomenon that we study here is how training on an alternate training distribution $\mathcal{D}' \neq \mathcal{D}$ can be *beneficial*, and lead to improved learning compared to training *on the same number of samples* from $\mathcal{D}$. We are particularly interested in how this can be achieved without changing the training algorithm, but by simply running the “standard” training algorithm, such as Stochastic Gradient Descent (SGD) on some standard architecture. This matches current training practices, where much of the innovation and competitive advantage is not from new training algorithms nor new models, but rather from finding good training distributions $\mathcal{D}'$. In this paper, we set out to provide a concrete framework and theoretical foundation for studying this phenomenon.

We emphasize the *computational* benefit of the distribution shift, where the main benefit of training on an alternative $\mathcal{D}'$ is not in reducing the number of training examples required information theoretically, but in allowing training to be *tractable* (e.g. possible in polynomial time)¹. We argue that this is the main benefit in practice, since sophisticated deep models are information theoretically learnable with sample complexity corresponding to the number of parameters, but also provably (worst case) computationally hard to learn. The real challenge in deep learning is therefore computational, and success rests on not being in this theoretical “worst case”. Our goal is to show that with an appropriate distribution shift, tractable learning is possible, even with SGD. Medvedev et al. (2025) recently introduced the no-

¹University of Chicago ²Toyota Technological Institute at Chicago ³University of Illinois Chicago ⁴Inria, DI/ENS, PSL ⁵Yale University ⁶Weizmann Institute of Science. Correspondence to: Marko Medvedev <medvedev@ttic.edu>, Idan Attias <idanattias88@gmail.com>.

tion of positive distribution shift for mixture distributions, showing that even with independent domains, mismatching training mixture proportions can improve test performance.

The specific form of distribution shift that we consider here is *covariate shift* in supervised classification, where the goal is to learn a predictor $h : \mathcal{X} \rightarrow \mathcal{Y}$ (with $\mathcal{Y} = \{0, 1\}$) under some target distribution $\mathcal{D}$ over $\mathcal{X}$, and we train using a different distribution $x \sim \mathcal{D}'$ but the same target conditional distribution $y|x$. We use $y|x = f(x)$ to denote this conditional distribution, where one can think of $f(x)$ as a random variable with a law determined by x .² We use $(x, y) \sim (\mathcal{D}, f)$ to denote the joint distribution over (x, y) specified by $x \sim \mathcal{D}, y = f(x)$. We are therefore interested in a predictor $\hat{h} = A(S)$ obtained by running some (possibly randomized) learning rule A on a training set $S \sim (\mathcal{D}', f)^m$ of m i.i.d. samples from $\mathcal{D}'$. The performance is measured by the error $L_{\mathcal{D},f}(\hat{h}) := \Pr_{(x,y) \sim (\mathcal{D},f)} [\hat{h}(x) \neq y]$, namely, the probability that $\hat{h}$ misclassifies an example drawn from the true target distribution $\mathcal{D}$. For a hypothesis class $\mathcal{H} \subseteq \mathcal{Y}^{\mathcal{X}}$, we denote by $L_{\mathcal{D},f}(\mathcal{H}) = \inf_{h \in \mathcal{H}} L_{\mathcal{D},f}(h)$ the minimal error achievable by a hypothesis in $\mathcal{H}$.

Our *Positive Distribution Shift (PDS)* learning framework is thus captured by the following definition.

Definition 1.1 (PDS Learning). A learning rule A **PDS learns** $(\mathcal{D}, f)$ and $\mathcal{H}$ using the training distribution $\mathcal{D}'$, with sample size $m(\epsilon)$ and runtime $T(\epsilon)$ if for every³ $\epsilon > 0$,

$$\mathbb{E}_{S' \sim (\mathcal{D}', f)^{m(\epsilon)}} [L_{\mathcal{D},f}(A(S'))] \leq L_{\mathcal{D},f}(\mathcal{H}) + \epsilon$$

and $A(S')$ runs in time at most $T(\epsilon)$.

What Definition 1.1 is missing, and we will make concrete in different ways in subsequent Sections, are quantifiers over the learning rule A and training distribution $\mathcal{D}'$, which are essential for discussing learning non-trivially.⁴

In this paper, we ask and formalize what it means to be able to learn a *hypothesis class* with “positive distribution shift”, and give examples of what is, and is not, tractably learnable in this sense. To this end, we consider several hypothesis classes which are (information theoretically easy but) computationally hard to learn in a standard PAC setting (i.e., no poly-time learning algorithm can ensure learning with matching target and training distributions). Ultimately, we would like to ask, and rigorously answer, whether such

²Formally, f specifies a conditional distribution law for $y|x$, and for every observed x_i , the corresponding label y_i is drawn independently from this conditional distribution—this is not a random function drawn once where $f(x_i)$ and $f(x_j)$ would then be dependent.

³For simplicity, we state learnability in expectation, however, all results hold with high probability, except for Theorem 4.5.

⁴Any single function f is trivially and meaninglessly “learnable” with hard-wired A that just always outputs f .

classes are tractably learnable with positive distribution shift (under some concrete formalization), by *training a typical neural network using standard SGD*. Rigorously proving learnability using standard SGD on standard networks remains mostly elusive⁵. Towards this goal, for different classes, we give evidence in the form of (a) proving PDS learnability using *some* tractable algorithm (but not SGD on a network); (b) proving PDS learnability using a “stylized” SGD on some specific network; and/or (c) showing PDS learnability experimentally using standard SGD on a standard network. See the Summary Section 6 for how we view and interpret PDS learning.

Realizability, label-noise, and agnostic learning setups.

In the *realizable* learning setting, the target function is deterministic and perfectly captured by the hypothesis class, that is, $f(x) = h^*(x) \in \mathcal{H}$ for some $h^* \in \mathcal{H}$, and $L_{\mathcal{D},f}(\mathcal{H}) = 0$. In the *random classification noise* setting (with binary labels), we assume the existence of some $h^* \in \mathcal{H}$ such that for every input x , $\Pr[f(x) = h^*(x) | x] = 1 - \eta$, where the noise rate $\eta \in [0, \frac{1}{2})$ may be known or unknown. In this case, $L_{\mathcal{D},f}(\mathcal{H}) = \eta$. In the general *agnostic* learning setting, we make no assumptions about the relationship between f and the hypothesis class, and the goal is to compete with the best possible hypothesis in $\mathcal{H}$. Our Positive Distribution Shift learning framework applies to all of these settings, although we primarily focus on the label-noise setting.

2. Warm-Up: Parities Are Efficiently Learnable With PDS

To illuminate how positive distribution shift (PDS) can help tractability, we first consider learning parity functions as a case study. We first review what is already known and then highlight our new perspective, which we later formalize as a result in Section 4. A parity is a Boolean function $\chi_S : \{\pm 1\}^d \rightarrow \{\pm 1\}$ defined as $\chi_S(x) = \prod_{i \in S} x_i$, where $S \subseteq [d]$ is a subset of the coordinates of size $k \leq d$ called the support. Under uniform input distribution $x \sim \text{Unif}\{\pm 1\}^d$, parities are *statistically* easy to learn: since there are $\binom{d}{k}$ candidate parities, so $O(k \log(d)) \leq \text{poly}(d)$ random samples suffice to identify the true support with high probability. Even in the presence of label noise, the sample complexity for *statistically* learning parities remains $\text{poly}(k \log d) \leq \text{poly}(d)$, and distribution shift does not help with this (and is not needed from an information-theoretical standpoint). *Computationally*, however, parities under the uniform distribution are hard to learn. In the *statistical query (SQ)* model parities are provably hard to learn, namely they require $d^{\Omega(k)}$ cost to learn with statistical queries (Kearns, 1998; Blum et al., 1994; Barak et al., 2001).⁵ Very few papers, if at all, truly analyze “standard” SGD, and even stylized SGD analysis is often technically complex.

2022). It is widely conjectured that in the PAC model, for any noise level $\eta > 0$, there is no algorithm that can learn k -sparse parities in time $\text{poly}(d, k, \epsilon)$ over uniform distribution with noise η , and even for $\log(d)$ -sparse parities there is no $\text{poly}(d, \epsilon)$ time algorithm (Kearns, 1998; Blum et al., 2003; Feldman et al., 2006; Applebaum et al., 2010; Goel et al., 2017).

However, training on a different distribution $\mathcal{D}' \neq \mathcal{D}$ can provide significant computational benefits, as it can reveal structure that is not visible under uniform inputs, which helps learning with respect to the starting distribution $\mathcal{D}$. For instance, consider training with a product distribution $\mathcal{D}'$ where all the bits have non-zero bias. For this training distribution $\mathcal{D}'$, coordinates x_i in the support $i \in S$ have much larger correlation with the target χ_S than the coordinates x_i not in the support $i \notin S$. This additional structure and correlations (that is not visible on the uniform distribution) gives a simple poly-time algorithm for learning parities: compute the correlation of each coordinate with the target and output the parity over those coordinates whose correlations are closer to this larger value, and this generalizes back to the starting distribution $\mathcal{D}$. This larger correlation with the target of coordinates in the support makes identifying the support tractably possible with gradient descent (GD) also, which picks up the support first and then makes the problem analogous to a linear predictor and hence tractably learnable by SGD on a neural network. In the terminology of (Abbe et al., 2023a), positive distribution shift makes the target function be a staircase in the Fourier basis of $\mathcal{D}'$, so PDS makes the function easy to learn even with SGD, and this generalizes back to the starting test distribution. This is our main focus: we want to understand what target functions are learnable with respect to a starting distribution $\mathcal{D}$ using SGD on a neural network with a shifted training distribution $\mathcal{D}'$.

Other authors have already considered the effect of input on tractability, but the view we take here is different in several ways. (Malach et al., 2021) looked at changing both the test and the training to the same distribution which is still under the standard PAC learning framework. They empirically demonstrate that in this standard PAC setting, training and testing on a distribution that is a mixture of uniform and all-bits-biased distribution, noiseless parities of sparsity $\log(d)$ are tractably learnable. Our perspective differs in that we are interested in keeping the test distribution while only modifying the training distribution. Some of our analysis parallels that of (Malach et al., 2021), but our view is different. (Cornacchia & Mossel, 2023) did look at keeping the same test distribution but their emphasis is on curriculum learning as an instance of positive distribution shift with computational benefit. They show that a curriculum where we train first on a randomly biased distribution and then on the uniform distribution enables learning parities with

respect to the uniform distribution. By contrast, we consider a single training distribution and argue that the essential factor is not the order of training (curriculum) but the overall training distribution $\mathcal{D}'$.

In Section 4, we present our formal results on PDS learning of parities. First, in Theorem 4.3, we show that there is a simple tractable PDS learning algorithm for learning any parity function. Secondly, we show that parities are PDS learnable with analyzable (i.e. with layerwise training) gradient descent on a standard feed-forward neural network. Finally, we empirically show that even dense parities are PDS learnable with standard gradient descent on a two-layer feed-forward ReLU network. This extends both the results Malach et al. (2021) and Cornacchia et al. (2025), since their results only consider sparse parities. For a more comprehensive discussion of related work, see Appendix A.

3. All Functions Are Easy, With the Right Training Distribution

In Section 2, we saw that a computationally hard class, namely noisy parities, is easy to learn with PDS. A natural question to ask is: *what classes are tractably PDS learnable? Is it possible to PDS learn all functions representable by neural networks, i.e., all circuits?* Note that even constant-depth circuits are hard in the PAC framework (Kharitonov, 1993; Daniely & Shalev-Shwartz, 2016; Daniely & Vardi, 2021). To answer these questions, we first introduce the most basic framework of positive distribution shift (PDS) learning, namely f-PDS learning. In this setting, the training distribution may depend not only on the test distribution $\mathcal{D}$ but also on the target function f (hence the name function-dependent PDS). The training points are sampled from such a distribution $\mathcal{D}'$, and the learner’s goal is to achieve low error on $\mathcal{D}$.

Definition 3.1 (f-Dependent Positive Distribution Shift (f-PDS)). A hypothesis class $\mathcal{H}$ is f-PDS learnable with a learning algorithm A with sample size $m(\epsilon)$ and runtime $T(\epsilon)$ if for every labeling rule $f : \mathcal{X} \rightarrow \Delta(\mathcal{Y})$ and every distribution $\mathcal{D}$ over $\mathcal{X}$, there exists a training distribution $\mathcal{D}'$ over $\mathcal{X}$ (allowed to depend on both $\mathcal{D}$ and f) such that for any $\epsilon > 0$ with $m(\epsilon)$ samples from $\mathcal{D}'$, A has runtime $T(\epsilon)$ and $\mathbb{E}_{S' \sim (\mathcal{D}', f)^{m(\epsilon)}} [L_{\mathcal{D}, f}(A(S')) - L_{\mathcal{D}, f}(\mathcal{H})] \leq \epsilon$.

The notion of f-PDS is very generous, perhaps even too generous as it allows us to “transmit” information about the target f through the choice of training distribution $\mathcal{D}'$. Indeed, as we show in the proof of Theorem 3.2 below, the learner can extract b bits encoded in the choice of $\mathcal{D}'$ using $O(b)$ samples, which allows specifying a hypothesis in a class of size $|\mathcal{H}| = 2^b$. Furthermore, this decoding can even be done by gradient descent on a neural network:

Theorem 3.2 (Any Poly-sized Circuit with Label Noise is

f-PDS Learnable with a Non-Standard Network). *For any $d, s \in \mathbb{N}$ there exists a fixed non-standard feed-forward neural network⁶, an initialization, and a learning rate, such that any function $f : \{0, 1\}^d \rightarrow \{0, 1\}$ representable by (i) a boolean circuit⁷ of size at most s , or (ii) a neural network of size at most s with $\text{polylog}(s)$ bits of precision, is f-PDS learnable with label noise by SGD on this neural network with samples and runtime $m(\epsilon), T(\epsilon) = \text{poly}(d, s, \frac{1}{\epsilon})$.*

For the proof of Theorem 3.2 see Appendix B. Note that the sample complexity and the number of steps do not depend on the noise because we encode f in $\mathcal{D}_f$. Encoding f in $\mathcal{D}_f$ amounts to a form of “cheating”, and the result rests on simulating an algorithms extracting the encoding from the distribution by gradient descent on a specially constructed and highly non-standard network, following Abbe & Sandon (2020); Abbe et al. (2021). If we allow such non-standard networks, the restriction to training by GD on a neural network becomes meaningless, and gradient descent on such a specially constructed network does something very different from gradient descent on more typical networks, allowing the network to extract information encoded in subtleties of the input distribution, completely ignoring the label, instead of trying to actually match the training labels. This is not what we are after. In order to try to understand whether the a target-function dependent $\mathcal{D}'$

It shows that the notion of f-PDS might not be restrictive enough, i.e. it allows us to take an impossible choice for $\mathcal{D}_f$ that we couldn’t be able to construct during training. It also makes the gradient descent variant considered here highly non-standard. Yet, Theorem 3.2 suggests the broader promise of f-PDS learning and naturally leads us to ask:

Open Question 3.3 (Universality of f-PDS Learnability). *Is every function over d inputs representable by a neural network of size s (i.e. any computable function) f-PDS learnable by standard gradient descent on a neural network with standard⁸ activation and initialization with runtime $\text{poly}(d, s)$ and with $\text{poly}(d, s)$ samples?*

This shifts the study about tractable learnability with neural networks to asking under what assumptions on the training distribution $\mathcal{D}'$ is the target PDS learnable.

⁶the network is non-standard in either its topology (i.e. not fully connected) and activation with standard initialization or it is fully connected with standard activation function but with special initialization.

⁷Boolean circuits: Let B be the set of gates {AND, OR, NOT}. A Boolean circuit C on d inputs is a finite directed acyclic graph (DAG) with input nodes $x_1, \dots, x_d$, internal nodes labeled by gates in B , and one output node. For $x \in \{0, 1\}^d$, values propagate along edges to define $f_C(x) \in \{0, 1\}$. The size of the circuit is the number of gate nodes.

⁸Here, standard initialization refers to sampling from a fixed, reasonable distribution (e.g., Gaussian) with parameters independent of the target function f .

4. A New Learning Framework: DS-PAC

In this section, we prefer to not have the training distribution $\mathcal{D}'$ depend on the target f , but rather allow $\mathcal{D}'$ to depend only on the hypothesis class $\mathcal{H}$ and the target distribution $\mathcal{D}$. We introduce two variants of such positive distribution shift: *deterministic*, where the training set is sampled directly from the training distribution, and *randomized*, where we first draw a distribution from a meta-distribution (a distribution over distributions) and then sample the training set from it.

Definition 4.1 (Deterministic Distribution-Shift PAC (D-DS-PAC)). A hypothesis class $\mathcal{H}$ is *Deterministic Distribution Shift PAC* (D-DS-PAC) learnable with a learning algorithm A with sample size $m(\epsilon)$ and runtime $T(\epsilon)$ if for every distribution $\mathcal{D}$ over $\mathcal{X}$ and labeling rule $f : \mathcal{X} \rightarrow \Delta(\mathcal{Y})$, there exists a training distribution $\mathcal{D}'$ over $\mathcal{X}$ (allowed to depend on $\mathcal{D}$ and $\mathcal{H}$ but not on f) such that for any $\epsilon > 0$, A has runtime $T(\epsilon)$ on $m(\epsilon)$ i.i.d. samples from $\mathcal{D}'$ and $\mathbb{E}_{S' \sim (\mathcal{D}', f)^{m(\epsilon)}} [L_{\mathcal{D}, f}(A(S')) - L_{\mathcal{D}, f}(\mathcal{H})] \leq \epsilon$. Formally, we say that a class $\mathcal{H}$ is D-DS-PAC learnable with label noise if this definition holds for a labeling rule that is induced by some $h^* \in \mathcal{H}$ with a label flipping noise $\eta \in [0, 1/2]$.

Definition 4.2 (Randomized Distribution Shift PAC (R-DS-PAC)). A hypothesis class $\mathcal{H}$ is *Randomized Distribution-Shift PAC* (R-DS-PAC) learnable with a learning algorithm A with sample size $m(\epsilon)$ and runtime $T(\epsilon)$ if for every distribution $\mathcal{D}$ over $\mathcal{X}$ and labeling rule $f : \mathcal{X} \rightarrow \Delta(\mathcal{Y})$, there exists a meta-distribution $\mathcal{M}_{\mathcal{D}}$ over distributions on $\mathcal{X}$ (allowed to depend on $\mathcal{D}$ and $\mathcal{H}$ but not on f) such that for any $\epsilon > 0$, A has runtime $T(\epsilon)$ on $m(\epsilon)$ i.i.d. samples from $\mathcal{D}' \sim \mathcal{M}_{\mathcal{D}}$ and $\mathbb{E}_{\mathcal{D}' \sim \mathcal{M}_{\mathcal{D}}} \mathbb{E}_{S' \sim (\mathcal{D}', f)^{m(\epsilon)}} [L_{\mathcal{D}, f}(A(S')) - L_{\mathcal{D}, f}(\mathcal{H})] \leq \epsilon$. Formally, we say that a class $\mathcal{H}$ is R-DS-PAC learnable with label noise if this definition holds for a labeling rule that is induced by some $h^* \in \mathcal{H}$ with a label flipping noise $\eta \in [0, 1/2]$.

In what follows, we study parities and juntas from the three perspectives on PDS learning: tractable, with analyzable GD, and empirical with standard GD.

Parities. For $S \subseteq \{1, \dots, d\}$, define the parity $\chi_S : \{\pm 1\}^d \rightarrow \{\pm 1\}$ as $\chi_S(x) = \prod_{i \in S} x_i$, where $x = (x_1, \dots, x_d) \in \{\pm 1\}^d$. The parity class over d bits is defined as $\text{Parity}_d = \{\chi_S : S \subseteq \{1, \dots, d\}\}$. We consider also k -sparse parities, $\text{Parity}_d^k = \{\chi_S : S \subseteq \{1, \dots, d\}, |S| = k\}$. In this part, we establish the following: (i) noisy parities are D-DS-PAC learnable with a tractable algorithm (Theorem 4.3); (ii) noisy parities are D-DS-PAC learnable with a slightly stylized, analyzable variant of gradient descent (where the initialization depends on the sparsity of the parity; Theorem 4.5). Moreover, we

show that noisy parities are f-PDS learnable using gradient descent on a feed-forward fully connected neural network for any test distribution, where for the uniform distribution such a result follows from (Daniely & Malach, 2020); (iii) empirically, standard gradient descent on a feed-forward neural network successfully learns noisy parities within the D-DS-PAC framework (Figure 1). This is in contrast to learning parity functions with noise in the PAC learning framework, which is believed to be computationally hard (Kearns, 1998; Blum et al., 2003; Applebaum et al., 2010; Feldman et al., 2006), both in general and even in the $\log(d)$ -sparse case.

Theorem 4.3 (Noisy Parities are Tractablely D-DS-PAC Learnable). *There is an efficient D-DS-PAC learning algorithm A that uses $\mathcal{D}' = \frac{1}{2}\text{Rad}(1 - \frac{1}{d})^{\otimes d} + \frac{1}{2}\text{Unif}(\{\pm 1\}^d)$ ⁹, such that for any distribution $\mathcal{D}$, the algorithm A D-DS-PAC learns the class of parity functions Parity_d with label noise $\eta < \frac{1}{2}$ using $m(\epsilon) = O\left(\frac{d^2 \log \frac{2d}{(1-2\eta)^2}}{(1-2\eta)^2}\right)$ samples from $\mathcal{D}'$ with runtime $T(\epsilon) = O\left(\frac{d^3 \log \frac{2d}{(1-2\eta)^2}}{(1-2\eta)^2}\right)$.*

Next, we show that parities are D-DS-PAC learnable with a stylized analyzable stochastic gradient descent (SGD) on a 2-layer network, namely with layerwise training, ℓ_1 regularization, and initializations that depend on the sparsity of the parity. The layerwise training analysis is standard in the theory of neural network learning literature (Abbe et al., 2023a; Barak et al., 2022; Bietti et al., 2022; Dandi et al., 2023), while the ℓ_1 regularization and the dependence of initialization on sparsity are used to make the approach more analyzable. Our experiments use standard joint training of both layers and with standard initialization and confirm the theoretical results. Note that, in contrast to f-PDS, here the training distribution may depend only on the class of parities (and the sparsity) but not on the target function itself. For the proofs of Theorem 4.3 and Theorem 4.5, see Appendix C.

Analyzable SGD 4.4 (Layerwise SGD with ℓ_1 -regularization). We perform stochastic gradient descent (SGD) on a two-layer network

$$f_{\text{NN}}(\mathbf{x}; \boldsymbol{\theta}) = \sum_{j \in [N]} a_j \text{ReLU}(\langle \mathbf{w}_j, \mathbf{x} \rangle + b_j),$$

where $\boldsymbol{\theta} = (\mathbf{a}, \mathbf{W}, \mathbf{b}) \in \mathbb{R}^{N(d+2)}$ are the parameters of the network. We do layerwise training, where we first train the bottom layer while holding the top layer fixed and then train the top layer while holding the bottom layer fixed, with square loss and with ℓ_1 regularization on the first layer with regularization parameter λ_1 with stepsize s_1 , and ℓ_2 regularization on second layer weights with stepsize s_2 . We initialize hyperparameters as $N = (4d + 6)$, $a_j^0 \in \{\pm 1\}$,

⁹A random variable has distribution $\text{Rad}(p)$ if it takes the value $+1$ with probability p and the value -1 with probability $1 - p$.

$\mathbf{w}_j^0 = \mathbf{0}$, and $b_j^0 \in \{-d - 1, -d, \dots, d, d + 1\}$, so that we have one for each combination of (a^0, b^0) ¹⁰.

Theorem 4.5 (Noisy Parities are D-DS-PAC Learnable with Stylized Analyzable GD). *For any fixed sparsity $k \leq d$, say $k = d/2$, of a parity function, the class Parity_d^k with label noise $\eta < \frac{1}{2}$ is D-DS-PAC learnable with Analyzable SGD 4.4 with λ and s that depend on k . For any distribution $\mathcal{D}$, if we train with $\mathcal{D}' = \frac{1}{2}\text{Rad}(1 - \frac{1}{d})^{\otimes d} + \frac{1}{2}\mathcal{D}_2$, where $\mathcal{D}_2$ samples l uniformly over $\{d, d - 2, \dots, -d\}$ then samples $\mathbf{x}$ uniformly conditioned on $\mathbf{1}^T \mathbf{x} = l$, using m samples and T steps of Analyzable SGD 4.4 with $m, T = O\left(\frac{d^9 \log(\frac{d}{\epsilon^2(1-2\eta)^2})}{\epsilon^3(1-2\eta)^6}\right)$, for any k sparse noisy parity target f we have $\mathbb{E}_{S' \sim (\mathcal{D}', f)^m} [L_{\mathcal{D}, f}(f_{\text{NN}}(\mathbf{x}; \boldsymbol{\theta}^{(T)}))] < \epsilon + \eta$.*

We empirically verify Theorem 4.5 using standard gradient descent on a two-layer fully connected ReLU network with standard initialization and training (Figure 1). For $\mu \in [-1, 1]$, let $\mathcal{D}_\mu$ be the distribution on $\{\pm 1\}^d$ with i.i.d. coordinates from $\text{Rad}(\frac{\mu+1}{2})$, so that $\mathcal{D}_0 = \text{Unif}\{\pm 1\}^d$. We study a degree- $k = 25$ parity over $d = 50$ bits with label noise, targeting the uniform distribution $\mathcal{D} = \mathcal{D}_0$. In the PDS setting, training samples come from $\mathcal{D}' = \frac{1}{2}\mathcal{D}_{0.96} + \frac{1}{2}\mathcal{D}_0$; in the standard setting from $\mathcal{D}$. Figure 1 (left) shows the evolution of the test error on $\mathcal{D}$ during training, where PDS yields significantly faster learning. The right panel compares training on $\mathcal{D}' = \mathcal{D}_0$ (left), $\mathcal{D}' = \frac{1}{2}\mathcal{D}_{0.96} + \frac{1}{2}\mathcal{D}_0$ (center), and $\mathcal{D}' = \mathcal{D}_{0.96}$, showing that only the mixture distribution (center), supports PDS generalization to the target $\mathcal{D}$.

f-PDS learnability of parities. f-PDS learnability of parities with analyzable GD with respect to the uniform test distribution follows from the existing result on learning parities with GD of Daniely & Malach (2020). Namely, they show that an analyzable version of gradient descent on 2-layer network learns parities with respect to the distribution $\mathcal{D}_1 = \frac{1}{2}\text{unif}(\{\pm 1\}^d) + \frac{1}{2}\mathcal{D}_S$, where $\mathcal{D}_S$ is uniform over inputs outside the support of the parity, S , and on S the inputs are all the same with probability $1/2$. This implies f-PDS learnability with respect to the uniform distribution (with training data still coming from $\mathcal{D}_1$, as needed in the definition of f-PDS). We further show that parities are *f-PDS learnable* (i.e. with respect to any test distribution $\mathcal{D}$) in Appendix C in Theorem C.4.

Juntas. For $k \leq d$, the k -junta class is $\text{Junta}_d^k = \{f : \{0, 1\}^d \rightarrow \{0, 1\} : \exists S \subseteq [d], |S| \leq k, \exists g : \{0, 1\}^S \rightarrow \{0, 1\} \text{ s.t. } f(\mathbf{x}) = g(\mathbf{x}_S)\}$, where $\mathbf{x}_S$ denotes the restriction of $\mathbf{x}$ to the coordinates in set S . In this part, we establish

¹⁰We do this for simplicity, but alternatively, we can take N polynomial in d and sufficiently large, and initialize hyperparameters as $a_j^0 \sim_{i.i.d} \text{Unif}(\{\pm 1\})$ and $b_j^0 \sim_{i.i.d} \text{Unif}(\{-d - 1, -d, \dots, d, d + 1\})$.

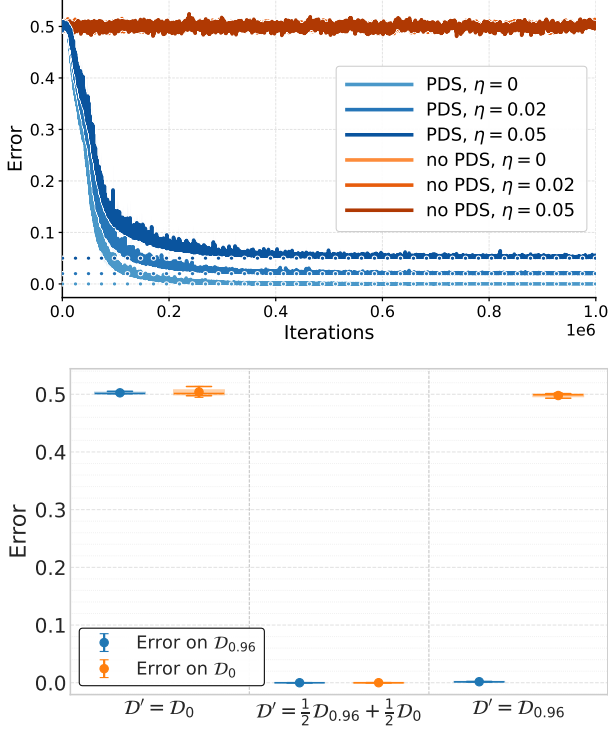


Figure 1. Noisy Parities. We study learning a degree-25 parity over 50 bits with label noise $\eta \in \{0, 0.02, 0.05\}$, using a two-layer ReLU network with 1024 hidden units trained by SGD (batch size 64, fresh samples, square loss, learning rate 0.01, both layers trained jointly). In the PDS setting, training samples are drawn from $\mathcal{D}' = \frac{1}{2}\mathcal{D}_{0.96} + \frac{1}{2}\mathcal{D}_0$, where $\mathcal{D}_\mu = \text{Rad}((\mu + 1)/2)^{\otimes d}$ for $\mu \in [-1, 1]$; in the standard setting, from $\mathcal{D}' = \mathcal{D}_0$. The top panel shows test error on $\mathcal{D}_0$ during training, where PDS yields markedly more efficient learning; dotted lines indicate Bayes error (η). The bottom panel compares $\mathcal{D}' = \mathcal{D}_0$, $\mathcal{D}' = \frac{1}{2}\mathcal{D}_{0.96} + \frac{1}{2}\mathcal{D}_0$, and $\mathcal{D}' = \mathcal{D}_{0.96}$, plotting test error on $\mathcal{D}_{0.96}$ (blue) and $\mathcal{D}_0$ (orange) after 10^6 steps. Only the mixture distribution achieves PDS generalization to the target $\mathcal{D}_0$.

the following: (i) noisy juntas are D-DS-PAC learnable by a Correlational Statistical Query (CSQ) algorithm (Theorem 4.6), (ii) noisy juntas are R-DS-PAC learnable with a stylized layerwise GD on a two-layer network with covariance loss, see Def. D.3 (Theorem 4.8), and (iii) empirically, noisy juntas are R-DS-PAC learnable with standard stochastic gradient descent with square loss on a feed-forward neural network (Figure 2). On the other hand, $\log(d)$ -juntas are believed to be hard to learn in the PAC model, even in the realizable case (Applebaum et al., 2010; Chandrasekaran & Klivans, 2025).

The algorithm in Theorem 4.6 is a Correlational Statistical Queries (CSQ) algorithm ((Ben-David et al., 1995; Bshouty & Feldman, 2002)), which are algorithms that access the data via queries $\phi : \mathbb{R}^d \rightarrow [-1, 1]$ and return $\mathbb{E}_{x,y}[\phi(x)y]$ up to some error tolerance τ .

Theorem 4.6 (Noisy Juntas are D-DS-PAC Learnable). *There exists an input distribution $\mathcal{D}'$ and an algorithm $\mathcal{A}$ such that for every target distribution $\mathcal{D}$ and any k , the class of k -sparse juntas Junta_d^k , with label noise $\eta < 1/2$, is D-DS-PAC learnable using $\mathcal{A}$ with $m = O(dk + 2^k)$ queries on $\mathcal{D}'$ of error tolerance $\tau = O((1 - 2\eta)2^{-k})$.*

We remark that a similar result can also be derived from Bshouty & Costa (2016) together with the connections between PDS and learning with membership queries established in Section 5 (see Appendix F for details). Our proof, however, shows D-DS-PAC learnability of noisy juntas directly by a different method, namely via correlation statistical queries, with an explicit construction of the positively shifted distribution $\mathcal{D}'$.

Now, we show that juntas are R-DS-PAC learnable with stylized analyzable GD.

Analyzable SGD 4.7 (Layerwise SGD with Covariance Loss). We perform gradient descent (SGD) on a two-layer network

$$f_{\text{NN}}(x; \theta) = \sum_{j \in [N]} a_j \text{ReLU}(\langle w_j, x \rangle + b_j),$$

where $\theta = (a, W, b) \in \mathbb{R}^{N(d+2)}$ are the parameters of the network. We do layerwise training, where we first train the bottom layer while holding the top layer fixed and then train the top layer while holding the bottom layer fixed, with covariance loss. The covariance loss is defined for a sample $\{(x_i, y_i)\}_{i \in [m]}$ and a predictor $\hat{f}(x)$ as $L_{\text{cov}}(x, y, \hat{f}) = (1 - cy\bar{y})(1 - y\hat{f}(x))_+$, where $\bar{y} = \frac{1}{m} \sum_{i \in [m]} y_i$ and c a positive constant such that $c \cdot \bar{y} < 1$. We initialize the first layer weights $w_{ij}^0 = 0$ for all $i \in [N]$, $j \in [d]$, and the second layer weights $a_i^0 = \kappa$, for $i \in [N/2]$ and $a_i^0 = -\kappa$, for $i \in [N/2]$, where $\kappa > 0$ is a constant. The biases are initialized to $b_i^0 = \kappa$, for all $i \in [N]$, and after the first step of training, $b_i^1 \stackrel{i.i.d.}{\sim} \text{Unif}[-L, L]$, where $L \geq \kappa$.

In the following, for $\mu \in [-1/2, 1/2]^{\otimes d}$, let $\mathcal{D}_\mu$ be the distribution on $\{\pm 1\}^d$ with coordinates $x_i \sim \text{Rad}((\mu_i + 1)/2)$, $i \in [d]$ (thus $\mathcal{D}_0 = \text{Unif}\{\pm 1\}^d$).

Theorem 4.8 (Noisy Juntas are R-DS-PAC Learnable with Stylized Analyzable GD). *For any fixed $k \leq d$, the class of k -sparse juntas Junta_d^k , with label noise $\eta < 1/2$, is R-DS-PAC learnable with respect to the uniform distribution $\mathcal{D}$ using Analyzable SGD 4.7. That is, there exists a meta distribution $\mathcal{M}_{\mathcal{D}'}$ defined as follows: a vector $\mu \sim \text{Unif}[-1/2, 1/2]^{\otimes d}$ is first sampled, and then $\mathcal{D}' = \frac{1}{2}\mathcal{D}_\mu + \frac{1}{2}\mathcal{D}_0$. For any $\epsilon > 0$ and noise level $\eta < 1/2$, after sampling $\mathcal{D}'$ from $\mathcal{M}_{\mathcal{D}'}$, Analyzable SGD 4.7 on a two-layer network of width $N = \tilde{\Omega}(\epsilon^{-1}(1 - 2\eta)^{-1})$, with $m = \tilde{\Omega}(d \log(1/\epsilon)\epsilon^{-2}(1 - 2\eta)^{-2})$ ¹¹ samples from $\mathcal{D}'$ and*

¹¹Here, for $c \in \mathbb{R}$, $\tilde{\Omega}(d^c) = \Omega(d^c \text{poly} \log(d))$.

$T = \tilde{\Omega}(\epsilon^{-2}(1-2\eta)^{-2})$ steps learns any k noisy junta target f with error $\mathbb{E}_{\mathcal{M}_D} \mathbb{E}_{S' \sim (\mathcal{D}', f)^m} [L_{(\mathcal{D}, f)}(f_{\text{NN}}(x; \theta^{(T)}))] < \eta + C_k \epsilon^{c_k}$, for constants $c_k, C_k > 0$ that depend only on k .

Theorem 4.8 builds on Cornacchia et al. (2025, Theorem 5), extending it to include label noise and our PDS training distribution, which is not a product measure. It remains an open question whether D-DS-PAC learnability can be established using GD. The proofs of Theorem 4.6 and Theorem 4.8 are in Appendix D.

Finally, we empirically verify Theorem 4.8 holds for standard gradient descent training on a two-layer fully-connected ReLU network in Figure 2. For $\mu \in [-1, 1]^{\otimes d}$, let $\mathcal{D}_\mu$ be the distribution on $\{\pm 1\}^d$ with coordinates $x_i \sim \text{Rad}((\mu_i + 1)/2)$, $i \in [d]$ (thus $\mathcal{D}_0 = \text{Unif}\{\pm 1\}^d$). For $k \in \{7, 9\}$, we consider learning the following k -juntas: $f_k(x) := \frac{1}{2} \prod_{i=1}^{k-2} x_i \cdot (1 + x_{k-1} + x_k - x_{k-1}x_k)$, with $x \in \{\pm 1\}^d$ and label noise, on the uniform target distribution (i.e. $\mathcal{D} = \mathcal{D}_0$). Thus, f_k is a k -junta with $\{\pm 1\}$ labels and minimum degree Fourier coefficient $k - 2$. In the PDS setting (R-DS-PAC), training uses samples from $\mathcal{D}' = \frac{1}{2}\mathcal{D}_\mu + \frac{1}{2}\mathcal{D}_0$, with $\mu \sim \text{Unif}[-1, 1]^{\otimes d}$, while in the standard setting (no PDS) it uses $\mathcal{D}$. In the left panel of Figure 2, we plot the evolution of the test error during training for learning f_9 on $d = 50$ bits, and observe that PDS yields far more efficient learning. In the right panel, we consider learning f_7 on varying input dimensions, and observe that the benefits of PDS increase with input dimension. We refer to Appendix G for the experiment’s details and additional plots.

We would like to explore PDS learning for additional well-studied hypothesis classes. We will now do so by connecting PDS learning to learning with membership queries.

5. DS-PAC and Membership Queries

In this section, we connect our PDS framework to the classical model of learning with membership queries (MQ). In this model, the learning algorithm can actively query specific points and obtain their labels from the target function f , in addition to sampling random labeled examples from the target distribution. The MQ model is strictly stronger than PAC (see Appendix A). Importantly, queries can be issued *adaptively*, that is, based on previously obtained labels, making it strictly more powerful than our “passive” PDS model, where examples are drawn non-adaptively from a training distribution.

The non-adaptive setting, defined below, has also been studied for specific hypothesis classes (in the realizable case), including parities and DNFs under the uniform distribution (Feldman, 2007), juntas (Bshouty & Costa, 2016), and decision trees of logarithmic depth in the input dimension (Bshouty, 2018).

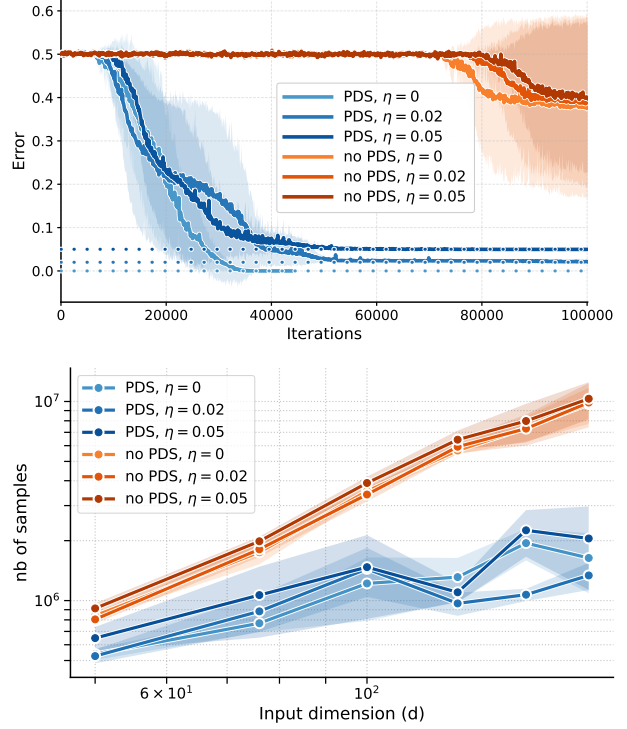


Figure 2. **Noisy juntas.** (Top) We consider learning f_9 (see Sec. 4) over $d = 50$ bits on $\mathcal{D} = \text{Unif}\{\pm 1\}^d$ with a two-layer ReLU network (1024 hidden units) trained with SGD (batch size 64, fresh samples, square loss, l.r. 0.01, both layers trained jointly). In the PDS setting we train on $\mathcal{D}' = \frac{1}{2}\mathcal{D}_\mu + \frac{1}{2}\mathcal{D}_0$, with $\mu \sim \text{Unif}[-1, 1]^{\otimes d}$ (and where $\mathcal{D}_\mu := \otimes_{i \in [d]} \text{Rad}((\mu_i + 1)/2)$), while in the standard (no PDS) setting $\mathcal{D}' = \mathcal{D}$. We plot the test error on $\mathcal{D}$ during training; the dotted lines show the Bayes error (i.e. η). (Bottom) We consider learning f_7 on $\mathcal{D} = \text{Unif}\{\pm 1\}^d$ and we plot the sample complexity needed to reach within 0.01 of Bayes error versus the input dimension. In both plots, PDS training is markedly more efficient.

Definition 5.1 (Non-Adaptive Membership Queries (NA-MQ)). A hypothesis class $\mathcal{H}$ over domain $\mathcal{X}$ is NA-MQ learnable with sample size $m(\epsilon)$ if there exist integers $m_{\text{rand}}(\epsilon)$ and $m_{\text{mq}}(\epsilon)$ with $m_{\text{rand}}(\epsilon) + m_{\text{mq}}(\epsilon) = m(\epsilon)$, a distribution $\mathcal{Q}$ over batches $\tilde{S}_{\mathcal{X}} \in \mathcal{X}^{m_{\text{mq}}(\epsilon)}$, and a learning algorithm A , such that the following holds. For every distribution $\mathcal{D}$ over $\mathcal{X}$ and every labeling rule $f : \mathcal{X} \rightarrow \Delta(\mathcal{Y})$, the distribution $\mathcal{Q}$ may depend on $\mathcal{D}$ and on known complexity parameters of $\mathcal{H}$, but not on the unknown target f and not on any observed labels. The learner has access to an i.i.d. labeled sample $S_{\text{rand}} = \{(x, f(x)) : x \sim \mathcal{D}\}$ of size $m_{\text{rand}}(\epsilon)$, and to a non-adaptive query batch $\tilde{S}_{\mathcal{X}} \sim \mathcal{Q}$ of size $m_{\text{mq}}(\epsilon)$ with labels $S_{\text{mq}} = \{(x, f(x)) : x \in \tilde{S}_{\mathcal{X}}\}$. The algorithm outputs $A(S_{\text{rand}}, S_{\text{mq}})$ such that $\mathbb{E}_{S_{\text{rand}} \sim (\mathcal{D}, f)^{m_{\text{rand}}(\epsilon)}, \tilde{S}_{\mathcal{X}} \sim \mathcal{Q}} [L_{\mathcal{D}, f}(A(S_{\text{rand}}, S_{\text{mq}})) - L_{\mathcal{D}, f}(\mathcal{H})] \leq \epsilon$. We say the learner is *deterministic NA-MQ*.

if $\mathcal{Q}$ is a point mass (the query set is fixed), and *randomized NA-MQ* otherwise.

Note that random classification noise is defined in the same way as in the PAC model, namely by flipping labels returned by f . As we show below, this non-adaptive model plays a central role in the DS-PAC framework. The proof is in Appendix E.

Theorem 5.2 (D-DS-PAC $\rightarrow$ NA-MQ). *For any hypothesis class $\mathcal{H}$, every D-DS-PAC-learning algorithm for $\mathcal{H}$ under label noise yields a NA-MQ-learning algorithm for $\mathcal{H}$ under label noise with the same sample complexity and running time.*

Theorem 5.3 (NA-MQ $\rightarrow$ R-DS-PAC). *Fix a hypothesis class $\mathcal{H}$ and a distribution $\mathcal{D}$ over $\mathcal{X}$. Suppose there is a (possibly randomized) non-adaptive MQ learner A for $\mathcal{H}$ under $\mathcal{D}$ that, using at most $m_0(\varepsilon)$ queries, outputs $\hat{h}$ with $\mathbb{E}[L_{\mathcal{D},f}(\hat{h})] \leq L_{\mathcal{D},f}(\mathcal{H}) + \varepsilon$. Then $\mathcal{H}$ is R-DS-PAC-learnable under $\mathcal{D}$ with sample size $m(\varepsilon) = O(m_0(\varepsilon/2)(\log m_0(\varepsilon/2) + \log(1/\varepsilon)))$. Moreover, if A is deterministic non-adaptive, then $\mathcal{H}$ is D-DS-PAC-learnable under $\mathcal{D}$ with the same sample bound.*

Moreover, we can show that a deterministic NA-MQ algorithm implies D-DS-PAC. As an immediate application, we obtain that DNFs under the uniform distribution and decision trees of logarithmic depth are R-DS-PAC learnable.

Natural open problems include the following: (i) Can DNFs and decision trees be learned with PDS by training a neural network with gradient descent? (ii) DNFs under the uniform distribution and log-depth decision trees are learnable in NA-MQ by identifying heavy Fourier coefficients. A natural broader question is whether the class of sparse functions, those with only a few non-zero Fourier coefficients, is also NA-MQ learnable under the uniform distribution using a number of queries polynomial in the dimension and the sparsity parameter, and hence R-DS-PAC learnable. This is known to hold in the adaptive MQ model (Mansour, 1994). (iii) Can we separate learning with adaptive MQ from NA-MQ/R-DS-PAC for natural classes? We note that Feldman (2007) sketches such a separation for certain artificial classes. For functions representable by polynomial-size circuits, we know that they are not R-DS-PAC learnable because constant-depth circuits are known to be hard to learn even with adaptive membership queries (Kharitonov, 1993).

6. Summary and Open Questions

Much of the effort in the current practice of ML is in “dataset selection” or finding the best training distribution $\mathcal{D}'$ in order to get good results on a target $\mathcal{D}$. It is important to develop a framework, language, methodology and theory to capture this. This paper is our attempt to make progress toward this goal.

In Section 3 on f-PDS, we asked which targets are learnable using some training $\mathcal{D}'$, with the f-PDS framework. The definition allows $\mathcal{D}'$ to depend on f , so this is not a “recipe” where one can construct $\mathcal{D}'_f$ during training. In particular, our Theorem 3.2 shows the existence of $\mathcal{D}'_f$ by “cheating” and leaking information through an unnatural and impossible choice of $\mathcal{D}'_f$, thus not providing insight but rather helping us refine the question to ask. This is done in Open Question 3.3 where we limit training to a fixed training rule that does not process “leaked” information: SGD on a regular neural net—the main contribution of this Section is thus in formulating Open Question 3.3.

The insight of PDS is two-fold:

- a) By understanding which training distributions $\mathcal{D}'_f$ make $(\mathcal{D}, f)$ learnable, we can gain insight into the properties of “good” training distributions. This perspective provides principled guidance for the dataset-selection problem. At present, such choices are often made in a largely heuristic manner, although there has been growing interest in empirical approaches to dataset selection and mixing (Ye et al., 2024; Liu et al., 2024; Xie et al., 2023; Shukor et al., 2025). We view PDS as providing theoretical guidance about what properties make a candidate training distribution effective. Our current understanding suggests that useful training distributions are those that more directly expose correlations between intermediate computations and the target function.
- b) PDS reframes the study of tractable learning with neural networks. Rather than asking which subclasses of neural-network-representable functions are tractably learnable (e.g., (Daniely & Vardi, 2020; Daniely et al., 2023)), or under which simple input distributions (e.g., uniform, Gaussian, etc.) learning is tractable (e.g., (Chen et al., 2022; Daniely & Vardi, 2021)), PDS asks a different question: for a target function f representable by a neural network and a target distribution $\mathcal{D}$, under what assumptions on the training distribution $\mathcal{D}'$ does efficient learning become possible? In this sense, rather than characterizing which neural-network-representable functions are tractably learnable, PDS seeks to characterize which training distributions make them tractably learnable.

In Sections 4 and 5 on DS-PAC and membership queries, we turned to a more prescriptive order of quantifiers, where here the training distribution $\mathcal{D}'$ depends only on the hypothesis class $\mathcal{H}$ and input distribution $\mathcal{D}$, but not on the target f . We present concrete definitions of a notion of learnability that we advocate studying, and initial results showing its power and depth. In particular, we show strong connections with membership query learning, and especially *non adaptive*

membership query learning. NA-MQ has only been sporadically studied in the past, and frequently not explicitly but by providing MQ methods that do not require adaptation. We hope this will reignite interest in NA-MQ, and more directly in DS-PAC learning.

If we allow an arbitrary tractable algorithm and to randomly choose a training distribution $\mathcal{D}'$, then DS-PAC is “equivalent” (with a small increase in sample complexity) to NA-MQ. With DS-PAC, unlike NA-MQ, we are mainly interested in using a particular learning rule, namely, SGD on a neural network. Our work raises the following open directions:

- *Are interesting hypothesis classes that are tractably learnable with NA-MQ (and thus also R-DS-PAC) also DS-PAC learnable using SGD on a neural network? E.g., decision trees?* An interesting candidate class of particular interest is the class of functions with a sparse Fourier decomposition.
- *Are sparse functions DS-PAC learnable? What about with using SGD on a neural network?* On the negative side, it would be good to show hardness of DS-PAC learning, which is equivalent to hardness of NA-MQ learning.
- *Are there natural classes that are MQ learnable, but not NA-MQ/DS-PAC learnable?* We are not aware of any work specifically on showing that adaptivity is necessary for MQ, i.e., separating A-MQ and NA-MQ for natural classes.
- *Is randomization necessary for DS-PAC learning?* In particular, are there concept classes that are learnable in the randomized DS-PAC model (R-DS-PAC) but not in the deterministic DS-PAC model (D-DS-PAC)? Moreover, does such a separation persist when the learner is restricted to SGD on a neural network? For example, are juntas D-DS-PAC learnable using SGD on a neural network?

Overall, we hope our work will create interest in PDS learning, which we view as an important learning paradigm. We can summarize the PDS hierarchy implied by our work as

$$\text{PAC} \not\Rightarrow \text{D-DS-PAC} \rightarrow \text{R-DS-PAC} \leftrightarrow \text{NA-MQ} \not\Rightarrow \text{A-MQ}.$$

Table 1 provides a summary of the current classification of hypothesis classes in the PDS hierarchy.

Class	Tractable Algorithm	“Analyzable” GD	Experiment (realistic GD)
Parity	D-DS-PAC Theorem 4.3	D-DS-PAC ^{GD} Theorem 4.5	Figure 1
k -Juntas	D-DS-PAC Theorem 4.6	R-DS-PAC ^{GD} Theorem 4.8	Figure 2
$\log(d)$ -depth Decision Trees	R-DS-PAC/NA-MQ (Bshouty, 2018)	?	?
DNF (uniform dist.)	R-DS-PAC/NA-MQ (Feldman, 2007)	?	?
Poly-Sized Circuits	f-PDS Theorem 3.2 (not R-DS-PAC)	f-PDS Theorem 3.2	?

Table 1. Learnability of hypothesis classes under random classification noise across multiple distribution shift frameworks. These classes are believed to be computationally hard to learn in the PAC model, or at least no polynomial-time algorithms are currently known. We ask whether they can instead be learned efficiently under positive distribution shift, using arbitrary tractable algorithms, provably analyzable gradient descent on neural networks, and empirically using standard gradient descent. For each hypothesis class, the test distribution is arbitrary, except for DNFs, where it is restricted to the uniform distribution. Notably, Theorem 4.8 was also established implicitly by Cornacchia et al. (2025). For the class of k -juntas, the algorithm runs in time polynomial in 2^k ; this dependence is unavoidable, since the sample complexity is $\Omega(2^k)$.

Acknowledgments

This work was done as part of the NSF-Simons funded collaboration on the Theoretical Foundations of Deep Learning (<https://deepfoundations.ai>). IA is supported by the National Science Foundation under Grant ECCS-2217023, through the Institute for Data, Econometrics, Algorithms, and Learning (IDEAL). EC was supported by the French government under management of Agence Nationale de la Recherche as part of the “Investissements d’avenir” program, reference ANR19-P3IA-0001 (PRAIRIE 3IA Institute). GV is supported by the Israel Science Foundation (grant No. 2574/25), by a research grant from Mortimer Zuckerman (the Zuckerman STEM Leadership Program), and by research grants from the Center for New Scientists at the Weizmann Institute of Science, and the Shimon and Golde Picker – Weizmann Annual Grant. NS was supported in part by the Simons Foundation and the NSF-TRIPOD Institute on Data Economics and Learning (IDEAL).

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. Its potential societal consequences are the standard ones associated with progress in machine learning, and we do not identify specific additional impacts that require separate discussion.

References

- Abbe, E. and Boix-Adsera, E. On the non-universality of deep learning: quantifying the cost of symmetry. *Advances in Neural Information Processing Systems*, 35: 17188–17201, 2022.
- Abbe, E. and Sandon, C. Poly-time universality and limitations of deep learning. *arXiv:2001.02992*, 2020.
- Abbe, E., Kamath, P., Malach, E., Sandon, C., and Srebro, N. On the power of differentiable learning versus PAC and SQ learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- Abbe, E., Adsera, E. B., and Misiakiewicz, T. The merged-staircase property: a necessary and nearly sufficient condition for sgd learning of sparse functions on two-layer neural networks. In *Conference on Learning Theory*, pp. 4782–4887. PMLR, 2022a.
- Abbe, E., Bengio, S., Cornacchia, E., Kleinberg, J., Lotfi, A., Raghu, M., and Zhang, C. Learning to reason with neural networks: Generalization, unseen data and boolean measures. *Accepted at NeurIPS 2022*, 2022b.
- Abbe, E., Adsera, E. B., and Misiakiewicz, T. SGD learning on neural networks: leap complexity and saddle-to-saddle dynamics. In *The Thirty Sixth Annual Conference on Learning Theory*, pp. 2552–2623. PMLR, 2023a.
- Abbe, E., Bengio, S., Lotfi, A., and Rizk, K. Generalization on the unseen, logic reasoning and degree curriculum. In *International Conference on Machine Learning*, pp. 31–60. PMLR, 2023b.
- Abbe, E., Cornacchia, E., and Lotfi, A. Provable advantage of curriculum learning on parity targets with mixed inputs. *Advances in Neural Information Processing Systems*, 36: 24291–24321, 2023c.
- Abbe, E., Cornacchia, E., Hazla, J., and Kougang-Yombi, D. Learning high-degree parities: The crucial role of the initialization. *arXiv preprint arXiv:2412.04910*, 2024.
- Aldous, D. and Vazirani, U. A markovian extension of valiant’s learning model. *Information and computation*, 117(2):181–186, 1995.
- Alman, J., Nadimpalli, S., Patel, S., and Servedio, R. A. Dnf learning via locally mixing random walks. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, pp. 2055–2061, 2025.
- Angluin, D. Learning regular sets from queries and counterexamples. *Information and computation*, 75(2):87–106, 1987.
- Angluin, D. Queries and concept learning. *Machine learning*, 2(4):319–342, 1988.
- Anil, C., Wu, Y., Andreassen, A., Lewkowycz, A., Misra, V., Ramasesh, V., Slone, A., Gur-Ari, G., Dyer, E., and Neyshabur, B. Exploring length generalization in large language models. *Advances in Neural Information Processing Systems*, 35:38546–38556, 2022.
- Applebaum, B., Barak, B., and Wigderson, A. Public-key cryptography from different assumptions. In *Proceedings of the forty-second ACM symposium on Theory of computing*, pp. 171–180, 2010.
- Arnaboldi, L., Dandi, Y., Krzakala, F., Pesce, L., and Stephan, L. Repetita iuvant: Data repetition allows SGD to learn high-dimensional multi-index functions. *arXiv preprint arXiv:2405.15459*, 2024.
- Arpe, J. and Mossel, E. Agnostically learning juntas from random walks. *arXiv preprint arXiv:0806.4210*, 2008.
- Back de Luca, A., Giapitzakis, G., and Fountoulakis, K. Learning to add, multiply, and execute algorithmic instructions exactly with neural networks. *Advances in Neural Information Processing Systems*, 38:78751–78795, 2025.

- Barak, B., Edelman, B. L., Goel, S., Kakade, S., Malach, E., and Zhang, C. Hidden progress in deep learning: Sgd learns parities near the computational limit. *arXiv preprint arXiv:2207.08799*, 2022.
- Bartlett, P. L., Fischer, P., and Höffgen, K.-U. Exploiting random walks for learning. In *Proceedings of the Seventh Annual Conference on Computational Learning Theory*, pp. 318–327, 1994.
- Ben Arous, G., Gheissari, R., and Jagannath, A. Online stochastic gradient descent on non-convex losses from high-dimensional inference. *The Journal of Machine Learning Research*, 22(1):4788–4838, 2021.
- Ben-David, S., Itai, A., and Kushilevitz, E. Learning by distances. *Information and Computation*, 117(2):240–250, 1995.
- Bietti, A., Bruna, J., Sanford, C., and Song, M. J. Learning single-index models with shallow neural networks. *Advances in Neural Information Processing Systems*, 35: 9768–9783, 2022.
- Blum, A., Furst, M., Jackson, J., Kearns, M., Mansour, Y., and Rudich, S. Weakly learning dnf and characterizing statistical query learning using fourier analysis. In *Proceedings of the twenty-sixth annual ACM symposium on Theory of computing*, pp. 253–262, 1994.
- Blum, A., Kalai, A., and Wasserman, H. Noise-tolerant learning, the parity problem, and the statistical query model. *Journal of the ACM (JACM)*, 50(4):506–519, 2003.
- Brutzkus, A., Daniely, A., and Malach, E. Id3 learns juntas for smoothed product distributions. In *Conference on Learning Theory*, pp. 902–915. PMLR, 2020.
- Bshouty, N. H. Exact learning via the monotone theory. In *Proceedings of 1993 IEEE 34th Annual Foundations of Computer Science*, pp. 302–311. IEEE, 1993.
- Bshouty, N. H. Exact learning from an honest teacher that answers membership queries. *Theoretical Computer Science*, 733:4–43, 2018.
- Bshouty, N. H. and Costa, A. Exact learning of juntas from membership queries. In *International Conference on Algorithmic Learning Theory*, pp. 115–129. Springer, 2016.
- Bshouty, N. H. and Feldman, V. On using extended statistical queries to avoid membership queries. *Journal of Machine Learning Research*, 2(Feb):359–395, 2002.
- Bshouty, N. H. and Haddad-Zaknoon, C. A. Adaptive exact learning of decision trees from membership queries. In *Algorithmic Learning Theory*, pp. 207–234. PMLR, 2019.
- Bshouty, N. H., Mossel, E., O’Donnell, R., and Servedio, R. A. Learning dnf from random walks. *Journal of Computer and System Sciences*, 71(3):250–265, 2005.
- Cagnetta, F. and Wyart, M. Towards a theory of how the structure of language is acquired by deep neural networks. *Advances in Neural Information Processing Systems*, 37: 83119–83163, 2024.
- Cagnetta, F., Petrini, L., Tomasini, U. M., Favero, A., and Wyart, M. How deep neural networks learn compositional data: The random hierarchy model. *Physical Review X*, 14(3):031001, 2024.
- Chandrasekaran, G. and Klivans, A. Learning juntas under markov random fields. *arXiv preprint arXiv:2506.00764*, 2025.
- Chen, S., Gollakota, A., Klivans, A., and Meka, R. Hardness of noise-free learning for two-hidden-layer neural networks. *Advances in Neural Information Processing Systems*, 35:10709–10724, 2022.
- Cornacchia, E. and Mossel, E. A mathematical model for curriculum learning for parities. In *International Conference on Machine Learning*, pp. 6402–6423. PMLR, 2023.
- Cornacchia, E., Mikulincer, D., and Mossel, E. Low-dimensional functions are efficiently learnable under randomly biased distributions. *arXiv preprint arXiv:2502.06443*, 2025.
- Damian, A., Lee, J., and Soltanolkotabi, M. Neural networks can learn representations with gradient descent. In *Conference on Learning Theory*, pp. 5413–5452. PMLR, 2022.
- Damian, A., Lee, J. D., and Bruna, J. The generative leap: Sharp sample complexity for efficiently learning gaussian multi-index models. *arXiv preprint arXiv:2506.05500*, 2025.
- Dandi, Y., Krzakala, F., Loureiro, B., Pesce, L., and Stephan, L. How two-layer neural networks learn, one (giant) step at a time. *arXiv preprint arXiv:2305.18270*, 2023.
- Dandi, Y., Troiani, E., Arnaboldi, L., Pesce, L., Zdeborová, L., and Krzakala, F. The benefits of reusing batches for gradient descent in two-layer networks: Breaking the curse of information and leap exponents. *arXiv preprint arXiv:2402.03220*, 2024.
- Dandi, Y., Pesce, L., Zdeborová, L., and Krzakala, F. The computational advantage of depth: Learning high-dimensional hierarchical functions with gradient descent. *arXiv preprint arXiv:2502.13961*, 2025.

- Daniely, A. and Malach, E. Learning parities with neural networks. *Advances in Neural Information Processing Systems*, 33:20356–20365, 2020.
- Daniely, A. and Shalev-Shwartz, S. Complexity theoretic limitations on learning dnf’s. In *Conference on Learning Theory*, pp. 815–830. PMLR, 2016.
- Daniely, A. and Vardi, G. Hardness of learning neural networks with natural weights. *Advances in Neural Information Processing Systems*, 33:930–940, 2020.
- Daniely, A. and Vardi, G. From local pseudorandom generators to hardness of learning. In *Conference on Learning Theory*, pp. 1358–1394. PMLR, 2021.
- Daniely, A., Srebro, N., and Vardi, G. Computational complexity of learning neural networks: Smoothness and degeneracy. *Advances in Neural Information Processing Systems*, 36:76272–76297, 2023.
- Deng, Y., Choi, Y., and Shieber, S. From explicit cot to implicit cot: Learning to internalize cot step by step. *arXiv preprint arXiv:2405.14838*, 2024. doi: 10.48550/arXiv.2405.14838. URL <https://arxiv.org/abs/2405.14838>.
- Feldman, V. Attribute-efficient and non-adaptive learning of parities and dnf expressions. *Journal of Machine Learning Research*, 8(7), 2007.
- Feldman, V., Gopalan, P., Khot, S., and Ponnuswami, A. K. New results for learning noisy parities and halfspaces. In *2006 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS’06)*, pp. 563–574. IEEE, 2006.
- Gamarnik, D. Extension of the pac framework to finite and countable markov chains. In *Proceedings of the twelfth annual conference on Computational learning theory*, pp. 308–317, 1999.
- Gerace, F., Saglietti, L., Sarao Mannelli, S., Saxe, A., and Zdeborová, L. Probing transfer learning with a model of synthetic correlated datasets. *Machine Learning: Science and Technology*, 3(1):015030, 2022.
- Glasgow, M. Sgd finds then tunes features in two-layer neural networks with near-optimal sample complexity: A case study in the xor problem. *arXiv preprint arXiv:2309.15111*, 2023.
- Goel, S., Kanade, V., Klivans, A., and Thaler, J. Reliably learning the relu in polynomial time. In Kale, S. and Shamir, O. (eds.), *Proceedings of the 2017 Conference on Learning Theory*, volume 65 of *Proceedings of Machine Learning Research*, pp. 1004–1042. PMLR, 2017.
- Goldman, S. A. and Kearns, M. J. On the complexity of teaching. *Journal of Computer and System Sciences*, 50(1):20–31, 1995.
- Goldman, S. A., Rivest, R. L., and Schapire, R. E. Learning binary relations and total orders. *SIAM Journal on Computing*, 22(5):1006–1034, 1993.
- Gopalan, P., Kalai, A. T., and Klivans, A. R. Agnostically learning decision trees. In *Proceedings of the fortieth annual ACM symposium on Theory of computing*, pp. 527–536, 2008.
- Ingrosso, A., Pacelli, R., Rotondo, P., and Gerace, F. Statistical mechanics of transfer learning in fully connected networks in the proportional limit. *Physical Review Letters*, 134(17):177301, 2025.
- Jackson, J. C. An efficient membership-query algorithm for learning dnf with respect to the uniform distribution. *Journal of Computer and System Sciences*, 55(3):414–440, 1997.
- Joshi, N., Misiakiewicz, T., and Srebro, N. On the complexity of learning sparse functions with statistical and gradient queries. *arXiv preprint arXiv:2407.05622*, 2024.
- Joshi, N., Koubbi, H., Misiakiewicz, T., and Srebro, N. Learning single-index models via harmonic decomposition. *arXiv preprint arXiv:2506.09887*, 2025.
- Kalai, A. T. and Teng, S.-H. Decision trees are pac-learnable from most product distributions: a smoothed analysis. *arXiv preprint arXiv:0812.0933*, 2008.
- Kalai, A. T., Samorodnitsky, A., and Teng, S.-H. Learning and smoothed analysis. In *2009 50th Annual IEEE Symposium on Foundations of Computer Science*, pp. 395–404. IEEE, 2009.
- Kearns, M. Efficient noise-tolerant learning from statistical queries. *Journal of the ACM (JACM)*, 45(6):983–1006, 1998.
- Kharitonov, M. Cryptographic hardness of distribution-specific learning. In *Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*, pp. 372–381, 1993.
- Klivans, A. and Meka, R. Moment-matching polynomials. *arXiv preprint arXiv:1301.0820*, 2013.
- Kou, Y., Chen, Z., Gu, Q., and Kakade, S. M. Matching the statistical query lower bound for k-sparse parity problems with stochastic gradient descent. *arXiv preprint arXiv:2404.12376*, 2024.

- Krause, M. and Lucks, S. Pseudorandom functions in and cryptographic limitations to proving lower bounds. *computational complexity*, 10(4):297–313, 2001.
- Kushilevitz, E. and Mansour, Y. Learning decision trees using the fourier spectrum. *SIAM journal on computing (Print)*, 22(6):1331–1348, 1993.
- Lee, J. D., Oko, K., Suzuki, T., and Wu, D. Neural network learns low-dimensional polynomials with SGD near the information-theoretic limit. *arXiv preprint arXiv:2406.01581*, 2024.
- Linial, N., Mansour, Y., and Nisan, N. Constant depth circuits, fourier transform, and learnability. *Journal of the ACM (JACM)*, 40(3):607–620, 1993.
- Liu, Q., Zheng, X., Muennighoff, N., Zeng, G., Dou, L., Pang, T., Jiang, J., and Lin, M. Regmix: Data mixture as regression for language model pre-training. *arXiv preprint arXiv:2407.01492*, 2024. doi: 10.48550/arXiv.2407.01492. URL <https://arxiv.org/abs/2407.01492>.
- Malach, E., Kamath, P., Abbe, E., and Srebro, N. Quantifying the benefit of using differentiable learning over tangent kernels. In *International Conference on Machine Learning*, pp. 7379–7389. PMLR, 2021.
- Mannelli, S. S., Ivashinka, Y., Saxe, A., and Saglietti, L. Tilting the odds at the lottery: the interplay of overparameterisation and curricula in neural networks. *arXiv preprint arXiv:2406.01589*, 2024.
- Mansour, Y. Learning boolean functions via the fourier transform. In *Theoretical advances in neural computation and learning*, pp. 391–424. Springer, 1994.
- Medvedev, M., Lyu, K., Li, Z., and Srebro, N. Shift is good: Mismatched data mixing improves test performance. *arXiv preprint arXiv:2510.25108*, 2025.
- Mignacco, F. and Mori, F. A statistical physics framework for optimal learning. *arXiv preprint arXiv:2507.07907*, 2025.
- Mossel, E. Deep learning and hierarchal generative models. *arXiv preprint arXiv:1612.09057*, 2016.
- Mossel, E., O’Donnell, R., and Servedio, R. A. Learning functions of k relevant variables. *Journal of Computer and System Sciences*, 69(3):421–434, 2004.
- Mousavi-Hosseini, A., Wu, D., Suzuki, T., and Erdogdu, M. A. Gradient-based feature learning under structured data. *Advances in Neural Information Processing Systems*, 36:71449–71485, 2023.
- Naor, M. and Reingold, O. Number-theoretic constructions of efficient pseudo-random functions. *Journal of the ACM (JACM)*, 51(2):231–262, 2004.
- O’Donnell, R. *Analysis of Boolean Functions*. Cambridge University Press, 2014. doi: 10.1017/CBO9781139814782.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- Power, A., Burda, Y., Edwards, H., Babuschkin, I., and Misra, V. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.
- Qharabagh, M. F., de Luca, A. B., Giapitzakis, G., and Fountoulakis, K. Learning to execute graph algorithms exactly with graph neural networks. In *International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2601.23207>.
- Roch, S. On learning thresholds of parities and unions of rectangles in random walk models. *Random Structures & Algorithms*, 31(4):406–417, 2007.
- Saglietti, L., Mannelli, S. S., and Saxe, A. An analytical theory of curriculum learning in teacher–student networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2022(11):114014, 2022.
- Schapire, R. E. and Sellie, L. M. Learning sparse multivariate polynomials over a field with queries and counterexamples. In *Proceedings of the sixth annual conference on Computational learning theory*, pp. 17–26, 1993.
- Shalev-Shwartz, S. and Ben-David, S. *Understanding machine learning: From theory to algorithms*. Cambridge university press, 2014.
- Shukor, M., Bethune, L., Busbridge, D., Grangier, D., Fini, E., El-Nouby, A., and Ablin, P. Scaling laws for optimal data mixtures. *arXiv preprint arXiv:2507.09404*, 2025. doi: 10.48550/arXiv.2507.09404. URL <https://arxiv.org/abs/2507.09404>.
- Şimşek, B., Bendjeddou, A., and Hsu, D. Learning gaussian multi-index models with gradient flow: Time complexity and directional convergence. *arXiv preprint arXiv:2411.08798*, 2024.
- Spielman, D. A. and Teng, S.-H. Smoothed analysis of algorithms: Why the simplex algorithm usually takes polynomial time. *Journal of the ACM (JACM)*, 51(3):385–463, 2004.

- Troiani, E., Dandi, Y., Defilippis, L., Zdeborová, L., Loureiro, B., and Krzakala, F. Fundamental limits of weak learnability in high-dimensional multi-index models. *arXiv preprint arXiv:2405.15480*, 2024.
- Wang, Z., Nichani, E., Bietti, A., Damian, A., Hsu, D., Lee, J. D., and Wu, D. Learning compositional functions with transformers from easy-to-hard data. *arXiv preprint arXiv:2505.23683*, 2025.
- Xie, S. M., Pham, H., Dong, X., Du, N., Liu, H., Lu, Y., Liang, P., Le, Q. V., Ma, T., and Yu, A. W. Doremi: Optimizing data mixtures speeds up language model pre-training. *arXiv preprint arXiv:2305.10429*, 2023. doi: 10.48550/arXiv.2305.10429. URL <https://arxiv.org/abs/2305.10429>.
- Ye, J., Liu, P., Sun, T., Zhan, J., Zhou, Y., and Qiu, X. Data mixing laws: Optimizing data mixtures by predicting language modeling performance. *arXiv preprint arXiv:2403.16952*, 2024. doi: 10.48550/arXiv.2403.16952. URL <https://arxiv.org/abs/2403.16952>.
- Zhou, H., Bradley, A., Littwin, E., Razin, N., Saremi, O., Susskind, J., Bengio, S., and Nakkiran, P. What algorithms can transformers learn? a study in length generalization. *arXiv preprint arXiv:2310.16028*, 2023.
- Zilles, S., Lange, S., Holte, R., Zinkevich, M., and Cesa-Bianchi, N. Models of cooperative teaching and learning. *Journal of Machine Learning Research*, 12(2), 2011.

A. Additional Related Work

Positive Distribution Shift. Medvedev et al. (2025) introduce the name *positive distribution shift* (PDS) for the phenomena where training on a distribution from test distribution can improve test performance. They focus on the case where test and train distributions are mixture distributions of K domains or tasks $\mathcal{D}_1, \dots, \mathcal{D}_K$, given by test and training mixing proportions $\mathbf{p} = (p_1, \dots, p_K)$ and $\mathbf{q} = (q_1, \dots, q_K)$, i.e. $\mathcal{D} = p_1\mathcal{D}_1 + \dots + p_K\mathcal{D}_K$ and $\mathcal{D}' = q_1\mathcal{D}_1 + \dots + q_K\mathcal{D}_K$. They consider the problem of finding the optimal train mixing proportions $\mathbf{q}^*$ for given fixed test mixing proportions $\mathbf{p}$ and fixed compute budget and identify for a number of scenarios $\mathbf{q}^*$ and the extend to which PDS can be beneficial. They also show that in this setting with independent domains or tasks, PDS almost always exists in the sense that it's almost always the case that $\mathbf{q}^* \neq \mathbf{p}$. Medvedev et al. (2025) constraints the distribution shift considered to just changing the mixing proportion, and their focus is on learning problems that were already tractable under initial distribution and PDS in their case reduces the number of samples needed to reach a certain error. In contrast, in our work we study PDS more broadly by introducing the various PDS learning frameworks and allow for any covariate shift. We focus on learning with SGD on a neural network and highlight the computational benefits of PDS in allowing the training to be tractable.

Empirical Procedures For Training Set Selections. There has been recent interest in empirical procedures for training-set-selections, such as choosing the best training distribution in the Pile dataset (Ye et al., 2024; Liu et al., 2024; Xie et al., 2023; Shukor et al., 2025). This line of work proposes various procedures for empirically finding (in our language) positive distribution shift. Another example of an empirical procedure for PDS is training with chain-of-thought but in a specific way to "internalize" it and make predictions without it (Deng et al., 2024).

GD on neural networks. A large body of work has analyzed the sample and time complexity of learning simple function classes—such as parities, juntas, and single/multi-index models—using stylized gradient descent on shallow networks under standard symmetric input distributions (e.g., uniform Boolean, standard Gaussian) (Ben Arous et al., 2021; Bietti et al., 2022; Abbe et al., 2023a; Dandi et al., 2023; 2024; Lee et al., 2024; Troiani et al., 2024; Arnaboldi et al., 2024; Şimşek et al., 2024; Damian et al., 2025; Joshi et al., 2025). For parities, it is known that under uniform Boolean inputs they are learnable by Stochastic Gradient Descent (SGD) with small batch size on an emulation network (Abbe & Sandon, 2020), but require at least $\Omega(\binom{d}{k})$ time in the Statistical Query (SQ) model (Kearns, 1998) or with GD under limited gradient precision (Abbe & Sandon, 2020). Positive results for gradient descent on standard shallow networks exist for sparse parities ($k = O_d(1)$), matching the SQ lower bound (Barak et al., 2022; Glasgow, 2023; Kou et al., 2024), and for dense parities depending on the initialization (Abbe & Boix-Adsera, 2022; Abbe et al., 2024). For juntas under uniform Boolean inputs, complexity is governed by the Fourier-Walsh structure of the target, captured by the leap for square loss (Abbe et al., 2022a;b; 2023a) and the SQ-leap for general losses (Joshi et al., 2024). Our work instead seeks to overcome these barriers via Positive Distribution Shift (PDS).

Beyond symmetric inputs, several works study structured data distributions—e.g., spike-covariance (Mousavi-Hosseini et al., 2023), hierarchical (Mossel, 2016; Cagnetta et al., 2024; Cagnetta & Wyart, 2024; Dandi et al., 2025), or non-centered product measures (Malach et al., 2021; Cornacchia et al., 2025)—showing that structure can reduce sample complexity relative to unstructured settings. In particular, Malach et al. (2021) analyze sparse parities under a mixture of uniform and biased distributions, but only for a differentiable model combining a linear predictor with an appended parity module, and with no label noise. By contrast, we consider standard two-layer architectures and demonstrate PDS benefits also for high-degree parities, and with label noise. For juntas, Cornacchia et al. (2025) show gains from training on randomly shifted inputs, but do not examine whether such favorable distributions can transfer to other target distributions, such as the uniform.

Other works have studied distribution shift between train and test, as we do here. A prominent line of research analyzes models tested outside their training domain (e.g., out-of-distribution, length generalization (Anil et al., 2022; Abbe et al., 2023b; Zhou et al., 2023; Abbe et al., 2022b; Power et al., 2022)), probing whether neural networks rely on reasoning or memorization. Transfer learning has also been examined as a way to cope with scarce target data (Damian et al., 2022; Gerace et al., 2022; Ingrosso et al., 2025). These works often treat distribution shift negatively—either as a stress test of extrapolation or as a necessity when labeled target data is limited. In contrast, we adopt a positive perspective, developing a unifying theory showing how a carefully chosen training distribution can actively facilitate learning. Closer to our approach, curriculum learning studies investigate training on a *sequence* of distributions of increasing complexity, and demonstrate benefits of ordering (Saglietti et al., 2022; Cornacchia & Mossel, 2023; Abbe et al., 2023c; Mannelli et al., 2024; Mignacco & Mori, 2025; Wang et al., 2025). By contrast, we focus on a *single* positively shifted training distribution and its ability to transfer to the target distribution, arguing that one well-chosen source can substantially ease learning.

Neural algorithmic reasoning via training data design. Complementary recent work in neural algorithmic reasoning studies an even stronger form of training-data design than the covariate-shift setting considered here: rather than drawing training samples from a helpful source distribution, the training set is explicitly constructed to encode local algorithmic instructions. In particular, [Back de Luca et al. \(2025\)](#) show that two-layer networks trained in the NTK regime can exactly execute binary permutations, addition, multiplication, and SBN instructions from a logarithmic number of instruction templates, and [Qharabagh et al. \(2026\)](#) extend this local-instruction paradigm to graph algorithms by training node-level MLPs and reusing them as a shared GNN update rule. These results are formulated from a different perspective than PDS: rather than analyzing performance with respect to a target distribution, they seek exact correctness on all valid inputs within a bounded domain. Nevertheless, they share the same underlying principle: carefully designed training data can substantially simplify learning.

Smoothed analysis. Smoothed analysis ([Spielman & Teng, 2004](#)) blends worst-case and average-case perspectives by measuring the maximum expected performance of an algorithm under slight random perturbations of its inputs. [Mossel et al. \(2004\)](#) showed that such perturbations alter the Fourier coefficients of Boolean functions and can drastically reduce complexity. Subsequent works established smoothed-analysis guarantees for learning juntas and decision trees under random product distributions ([Kalai & Teng, 2008](#)), and for DNFs ([Kalai et al., 2009](#)). The ID3 decision-tree algorithm was later shown to efficiently learn juntas in the smoothed model ([Brutzkus et al., 2020](#)), and these ideas were extended to general Markov Random Fields with a smoothed external field ([Chandrasekaran & Klivans, 2025](#)). Related work also studied smoothing via additive Gaussian noise in the Gaussian input setting ([Klivans & Meka, 2013](#)). Our work departs from this literature in two key ways: 1) Positive Distribution Shift (PDS) involves deliberate, often large and typically deterministic shifts (except in the case of R-DS-PAC), rather than small random perturbations; and 2) we focus on providing quantitative computational guarantees for concrete algorithms, including gradient descent on neural networks.

Learning with (adaptive) membership queries. Classical results in this model established its power relative to the standard PAC framework. Angluin’s foundational work showed that finite automata can be exactly learned from queries and counterexamples ([Angluin, 1987](#)), and that monotone DNF formulas are efficiently learnable with membership queries ([Angluin, 1988](#)). Subsequent work extended these results to richer classes: [Bshouty \(1993\)](#) gave polynomial-time algorithms for decision trees, while [Schapire & Sellie \(1993\)](#) developed methods for learning sparse multivariate polynomials over $\text{GF}(2)$. Fourier-analytic techniques played a central role in later breakthroughs: [Linial et al. \(1993\)](#) showed that constant-depth circuits (AC^0) can be efficiently learned from membership queries using low-degree Fourier concentration, and [Kushilevitz & Mansour \(1993\)](#) applied similar ideas to decision trees under the uniform distribution. Building on this line, [Jackson \(1997\)](#) obtained an algorithm for learning DNF formulas with membership queries under the uniform distribution.

Learning from random walks. The passive learning model via random walks ([Aldous & Vazirani, 1995](#); [Bartlett et al., 1994](#); [Gamarnik, 1999](#); [Roch, 2007](#)) lies strictly between uniform-distribution learning and the membership query model: it is weaker than the latter, yet stronger than the former. This model has enabled several important results, including learning DNFs and decision trees with random classification noise under the uniform distribution ([Bshouty et al., 2005](#)), learning parities with noise (as a consequence of ([Bshouty & Feldman, 2002](#); [Bshouty et al., 2005](#))), and agnostic learning of juntas ([Arpe & Mossel, 2008](#)). Similar to the membership query setting, random classification noise can be tolerated whenever there exists an algorithm for the realizable case. In the context of this paper, the random walk model can be viewed as a specific case of R-DS-PAC.

B. Proofs for f-PDS

Proof of Theorem 3.2. If we have freedom to change the network architecture, activation, initialization, and learning rate, we can use the PAC-universality of neural networks to show Theorem 3.2.

Note that it suffices to show that we can transmit $\text{poly}(d)$ bits by encoding them into a distribution $\tilde{\mathcal{D}}$. We can use this to decode the network using a PAC learning algorithm. Simulating this decoding algorithm with GD on the non-standard network gives us the desired learning algorithm.

We first show the subroutine of this PAC algorithm for transmitting bits using a distribution.

Lemma B.1 (Transmitting r bits using a distribution). *Let θ be a set of $r = \text{poly}(d) < 2^{d-1}$ elements from $\{0, 1\}^d$. There exists a $\text{poly}(m, d)$ procedure **DECODE** such that for any d and any $r \leq \text{poly}(d)$ there are a distribution $\mathcal{D}$ over $\{0, 1\}^d$*

and $m \leq \text{poly}(r, d, \log \frac{1}{\epsilon})$ such that for $S = \{x_i\}_{i=1}^m \sim (\mathcal{D}, f)^m$ and $\hat{\theta} = \text{DECODE}(S)$ we have $\hat{\theta} = \theta$ with probability at least $1 - \epsilon$.

Proof of Lemma B.1. Associate each $x \in \{0, 1\}^d$ with an integer $0, 1, \dots, 2^d - 1$. Define $\mathcal{D}$ as follows: for $i = 1, \dots, r$, $P(x = i) = \begin{cases} \frac{2}{5r}, & \text{if } \theta[i] = 0 \\ \frac{3}{5r}, & \text{if } \theta[i] = 1. \end{cases}$. For $i > r$, $P(x = i) = 0$, and $P(x = 0) = 1 - \sum_{i \neq 0} P(x = i)$. We will need $O\left(\frac{r(\log r + \log \frac{1}{\epsilon})}{(\frac{1}{r})^2}\right) = O(r^3(\log r + \log \frac{1}{\epsilon}))$ samples from $\mathcal{D}$. Let $\hat{P}$ be the empirical distribution we get on $\{0, 1\}^d$ from the sample $S = (x_1, \dots, x_m)$. With this many samples, we have that with probability ≥ 0.99 , we have $\hat{P}(x = i) \in (P(x = i) - \frac{1}{20r}, P(x = i) + \frac{1}{20r})$, so we can set the threshold $\hat{P}(x = i) = \frac{2.5}{5r}$ to recover the bit. The runtime of this procedure is $O(mrd) = \text{poly}(r, \log \frac{1}{\epsilon})$. To see why this many samples are sufficient, let $Y_i = \sum_{j=1}^m \frac{1_{x_j=i}}{m}$. Note that $\mathbb{E}(1_{x_i=i}) = P(x = i)$, so by Hoeffding inequality applied to $Y_i - P(x = i)$ with $0 \leq \frac{1_{x_j=i}}{m} \leq \frac{1}{m}$

$$P\left(\cap_{i=1}^r |Y_i - P(x = i)| < \frac{1}{20r}\right) \geq 1 - 2r \exp\left(-\frac{\frac{1}{20^2 r^2}}{m \frac{1}{m^2}}\right) = 1 - 2r \exp\left(-\frac{m}{20^2 r^2}\right).$$

Therefore, if we recover bits using $\hat{\theta}_i = \begin{cases} 1 & \text{if } Y_i \geq \frac{3}{5r} \\ 0 & \text{if } Y_i < \frac{3}{5r} \end{cases}$, then with the same probability as above we recover the bits correctly, $P(\hat{\theta} = \theta) \geq 1 - 2r \exp(-\frac{m}{20^2 r^2})$. Taking $m = O(r^2(\log r + \log \frac{1}{\epsilon}))$ suffices to have $2r \exp(-\frac{m}{20^2 r^2}) \leq \epsilon$, i.e. to transmit the bits with probability $\geq 1 - \epsilon$. $\square$

Consider learning without noise first. We will add the label noise later.

Encoding the network. Let $f : \{0, 1\}^d \rightarrow \{0, 1\}$ be boolean circuit satisfying of poly size. Then f can be encoded in $r = \text{poly}(d)$ atoms of $\{0, 1\}^d$.

For the circuits (i) we use the following encoding: we encode the adjacency matrix of size $M \times M$ of the DAG representing the boolean circuit, where $M = \text{poly}(d)$ is the total number of nodes of the DAG, with the value at entry (i, i) states whether the node is input, output, or which of the gate node types, and the value at (i, j) entry represents if there is an edge and if yes what direction the edge is between node i and node j . Note that from the adjacency matrix we can reconstruct the function in time $\text{poly}(d)$. So, for elements $(x^1, \dots, x^d) \in \{0, 1\}^d$ we encode

- Use the first $\log \text{poly}(d)$ bits to encode the the size of the circuit, M , and the total number of bits needed to transmit, r
- Use the next $2 \log \text{poly}(d)$ bits to encode the value of the coordinates i and j of the entry (i, j) in the adjacency matrix.
- Use the next 4 bits to encode the value of the entry.
- Use the remaining bits to mark that the atom is coming from this distribution by setting them to 1, note that there is at least cn of these for some c (that can be taken to be $\frac{1}{4}$ for large enough n).

For poly-sized networks we use the following encoding: We can do the encoding similarly: (i) Use the first $3 \log \text{poly}(d)$ bits to encode the depth L , width W , and total number of bits r , (ii) Use the next $3 \log \text{poly}(d)$ bits to location of the weight in the format (i, j) , which represents depth and the width of that layer in which the weight is located, (iii) use the next $d/2$ bits encode the weight value (e.g. if we want rational weights, we can use the first $d/4$ bits for the numerator and the next $d/4$ bits for the denominator, (iv) use the remaining bits to mark that the atom is coming from this distribution by setting them to 1, note that there is at least cn of these for some c (that can be taken to be $\frac{1}{4}$ for large enough n).

Note that the width and the depth are encoded in all $x \in \theta$ (and if the whole layer is missing, we can tell from other points). So, similarly, in the case that we do not recover θ , we can infer that we didn't recover it by counting the number of distinct weight positions recovered. In this case, we will return the zero predictor.

The rest of the proof is analogous for circuits and networks. .

Since there is $\text{poly}(d)$ gates, this encoding procedure requires $r = \text{poly}(d)$ elements $x_i \in \{0, 1\}^d$.

Learning Algorithm For f-PDS. Call the set of r elements $x_i \in \{0, 1\}^n$ that encode the circuit θ . Now using Lemma B.1, there is a distribution $\mathcal{D}_\theta$ and a PAC learning algorithm $\mathcal{A}$ which using $m = O(r^2(\log r + \log \frac{1}{\epsilon}))$ samples S from $S \sim \mathcal{D}_\theta^m$ recovers $\mathcal{A}(S) = \hat{\theta}$ such that $\hat{\theta} = \theta$ with probability at least $1 - \epsilon$.

With probability ϵ it happens that we do not recover all of θ . Note that by the construction of $\mathcal{D}_\theta$ from Lemma B.1, any $x \sim \mathcal{D}_\theta$ from the support of this distribution is in θ . Note that the size of the adjacency matrix is encoded in any of the elements in the support of $\mathcal{D}_\theta$, so we can tell if do not recover θ . In this case, we will return the zero predictor.

This gives a PAC learning algorithm $\mathcal{A}'$ to recover f using $m = O(r^2(\log r + \log \frac{1}{\epsilon}))$ samples from $\mathcal{D}_\theta$ as described above,

$$\mathcal{A}'(S) = \begin{cases} \mathcal{A}(S) & \text{if we recover all of } \theta \\ \hat{f} \equiv 0 & \text{if we don't recover } \theta. \end{cases}$$

Note that with $m' := 4m + \log(1/\epsilon) = O(r^2(\log r + \log \frac{1}{\epsilon}))$ samples from

$$\mathcal{D}_f := \frac{1}{2}\mathcal{D}_\theta + \frac{1}{2}\mathcal{D}$$

the algorithm $\mathcal{A}'$ has the same output with probability at least $1 - 2\epsilon$, i.e. recovers f , and with probability 2ϵ returns 0. This implies that the square loss of $\mathcal{A}'$ over any test distribution is at most 2ϵ , so the square loss over $\mathcal{D}_f$ is at most 2ϵ as well. We can rename here $\epsilon = \frac{\epsilon}{2}$.

At this point we keep the labels noiseless. Let $\bar{\mathcal{D}}$ denote the distribution over (x, y) where $x \sim \mathcal{D}$ and $y = f(x)$, and let $\bar{\mathcal{D}}_f$ denote the distribution over (x, y) where $x \sim \mathcal{D}_f$ and $y = f(x)$. Since $\mathcal{A}'$ exactly learns f with probability at least $1 - \epsilon$ and otherwise returns the zero predictor, for every marginal test distribution $\mathcal{Q}$ over $\{0, 1\}^d$ we have

$$\mathbb{E}_{S \sim (\bar{\mathcal{D}}_f, f)^{m'}} \left[\mathbb{E}_{x \sim \mathcal{Q}} (\mathcal{A}'(S)(x) - f(x))^2 \right] \leq \epsilon.$$

In particular, this holds for $\mathcal{Q} = \mathcal{D}_f$ and for $\mathcal{Q} = \mathcal{D}$.

Note that $\mathcal{A}'$ is a tractable PAC learning algorithm for learning f on the noiseless distribution $\bar{\mathcal{D}}_f$. More importantly, $\mathcal{A}'$ is an f-PDS learning algorithm for learning f on $\bar{\mathcal{D}}$ with samples from $\bar{\mathcal{D}}_f$, since $\mathcal{A}'$ exactly learns f with probability $1 - \epsilon$ and returns 0 otherwise. Also, $\mathcal{A}'$ does not use the labels, so its output distribution is unchanged if the labels in its training samples are corrupted by label noise.

We summarize the noiseless f-PDS result in the following lemma and then focus on simulating this with GD on a non-standard network.

Lemma B.2 (f-PDS Tractable Algorithm For Learning Any Circuit). *Let $f : \{0, 1\}^d \rightarrow \{0, 1\}$ be a function that satisfies either (i) f is a circuit of any depth and of size $s = \text{poly}(d)$ OR (ii) f is a fully connected feed forward ReLU network of size $s = \text{poly}(d)$ bit precision $\frac{d}{2}$. In both cases (i) and (ii), there exists an f-PDS algorithm $\mathcal{A}$ such that for any $f : \{0, 1\}^d \rightarrow \{0, 1\}$ satisfying either (i) or (ii), and for any $\mathcal{D}$, there exists a distribution $\mathcal{D}_f$ so that $\mathcal{A}$ learns f over the noiseless distribution $\bar{\mathcal{D}}$ with expected square loss at most ϵ in $m = \text{poly}(d, \frac{1}{\epsilon})$ samples and runtime $T = \text{poly}(d, \frac{1}{\epsilon})$ with samples from $\bar{\mathcal{D}}_f$. Moreover, $\mathcal{A}$ ignores labels, so the same output distribution is obtained under label noise in the training samples.*

Note that both in the case of circuits and networks, the runtime of $\mathcal{A}'$ is $\text{poly}(d, \frac{1}{\epsilon})$.

Simulating with Gradient Descent. Now we will use the result on simulating PAC learning algorithm with non-standard gradient descent on neural networks to show that $\mathcal{A}'$ can also be turned into a gradient descent based algorithm. We will use the result Theorem 1a (for mini-batch stochastic GD) from (Abbe et al., 2021) or equivalently Theorem 17 and Remark 17 in (Abbe & Sandon, 2020) for the simulation. The theorem shows that for any dimension d , runtime T , and q bits of randomness, and sample size m of a PAC learning algorithm, there exists a set of neural nets $\mathcal{N}$ that depend on d and the runtime $T = \text{poly}(d)$, with p' parameters, an initialization with $q' = r + O(T \log b)$ bits of randomness, poly-time computable activation functions σ , and some stepsize γ , so that for training using $\rho < \min\{1/8b, 1/12\}$ approximate gradients over batches of size b on the neural networks from this set $\mathcal{N}$ with stepsize γ the following claim holds. For any $\delta > 0$ and $p' = \text{poly}(d, m, r, T, \rho^{-1}, b, \delta^{-1})$ with $T' = O(mn/\delta)$, if any such PAC learning algorithm $\mathcal{A}$ learns a distribution $\mathcal{Q}$ with square loss ϵ , then there is a neural net in this set that learns $\mathcal{Q}$ with square loss at most $\epsilon + \delta$.

Taking $\mathcal{A}$ to be the previously described PAC learning algorithm for f-PDS $\mathcal{A}'$, and taking $\mathcal{Q} = \bar{\mathcal{D}}_f$, there exists a fixed neural network $N \in \mathcal{N}$ such that running mini-batch SGD on this network learns f with respect to $\bar{\mathcal{D}}_f$ with square loss $\epsilon + \delta$. Taking $\epsilon = \frac{\epsilon}{4}$ and $\delta = \frac{\epsilon}{4}$, $b = 1$, and $\rho = \frac{1}{d}$, we have that for

$$p' = \text{poly}(d, \frac{1}{\epsilon}), \quad T' = O\left(\frac{r^2(\log r + \log \frac{1}{\epsilon})d}{\epsilon}\right),$$

and

$$r' = O\left(\frac{r^2(\log r + \log \frac{1}{\epsilon})d}{\epsilon}\right),$$

the network $N \in \mathcal{N}$ achieves square loss at most $\epsilon/2$ on learning f over $\bar{\mathcal{D}}_f$, i.e.

$$\mathbb{E}_{S \sim (\bar{\mathcal{D}}_f, f)^m} \left[\mathbb{E}_{x \sim \mathcal{D}_f} (N(S)(x) - f(x))^2 \right] \leq \epsilon/2,$$

where the expectation is over the initialization and the mini-batches from $\bar{\mathcal{D}}_f$.

In the theorem that we used, the training and testing distributions are the same and equal to $\bar{\mathcal{D}}_f$, so we want to bound the error of the neural network N on $\bar{\mathcal{D}}$ based on its error on $\bar{\mathcal{D}}_f$. Since $\mathcal{D}_f = \frac{1}{2}\mathcal{D} + \frac{1}{2}\mathcal{D}_\theta$, we can write

$$\begin{aligned} \mathbb{E}_{S \sim \bar{\mathcal{D}}_f^m} \left[\mathbb{E}_{x \sim \mathcal{D}_f} (N(S)(x) - f(x))^2 \right] &= \frac{1}{2} \mathbb{E}_{S \sim (\bar{\mathcal{D}}_f, f)^m} \left[\mathbb{E}_{x \sim \mathcal{D}} (N(S)(x) - f(x))^2 \right] \\ &\quad + \frac{1}{2} \mathbb{E}_{S \sim (\bar{\mathcal{D}}_f, f)^m} \left[\mathbb{E}_{x \sim \mathcal{D}_\theta} (N(S)(x) - f(x))^2 \right]. \end{aligned}$$

The second term is nonnegative. Therefore,

$$\mathbb{E}_{S \sim (\bar{\mathcal{D}}_f, f)^m} \left[\mathbb{E}_{x \sim \mathcal{D}} (N(S)(x) - f(x))^2 \right] \leq 2 \mathbb{E}_{S \sim (\bar{\mathcal{D}}_f, f)^m} \left[\mathbb{E}_{x \sim \mathcal{D}_f} (N(S)(x) - f(x))^2 \right] \leq \epsilon.$$

Now define the final classifier by thresholding the network output:

$$\hat{y}_S(x) = \mathbf{1}\{N(S)(x) \geq 1/2\}.$$

Because $f(x) \in \{0, 1\}$, whenever $\hat{y}_S(x) \neq f(x)$ we have

$$(N(S)(x) - f(x))^2 \geq \frac{1}{4}.$$

Hence, pointwise,

$$\mathbf{1}\{\hat{y}_S(x) \neq f(x)\} \leq 4(N(S)(x) - f(x))^2.$$

Taking expectations gives

$$\mathbb{E}_{S \sim (\bar{\mathcal{D}}_f, f)^m} [L_{\bar{\mathcal{D}}, f}(\hat{y}_S)] \leq 4\epsilon.$$

Finally, add label noise. Let $\mathcal{D}^\eta$ denote the distribution obtained from $\bar{\mathcal{D}}$ by corrupting the label with probability at most η . Since

$$\mathbf{1}\{\hat{y}_S(x) \neq y_{\text{noisy}}\} \leq \mathbf{1}\{\hat{y}_S(x) \neq f(x)\} + \mathbf{1}\{y_{\text{noisy}} \neq f(x)\},$$

we have

$$\mathbb{E}_{S \sim (\mathcal{D}^\eta, f)^m} [L_{\mathcal{D}^\eta, f}(\hat{y}_S)] \leq 4\epsilon + \eta.$$

Here we used that $\mathcal{A}'$ ignores labels, so the same learned predictor distribution is obtained whether the labels in the training samples from $\mathcal{D}_f$ are noiseless or corrupted.

Thus the thresholded network has 0-1 error at most $4\epsilon + \eta$ under label noise. Equivalently, to state the theorem with error $\epsilon + \eta$, run the above argument with the internal square-loss parameter $\epsilon/4$. The sample complexity and runtime remain $\text{poly}(d, s, \frac{1}{\epsilon})$.

This finishes the proof.

Remark Note that we can extend the set of allowed functions to include all $f : \{0, 1\}^d \rightarrow \{0, 1\}$ functions that can be encoded on $\text{poly}(d)$ bits of $\{0, 1\}^d$. For example, this set includes all short description functions. $\square$

C. Learning parity functions

Proof of Theorem 4.3. Let S be the support of the parity. Recall that under random classification noise $y = \xi \chi_S(\mathbf{x})$ with $\xi = \text{Rad}(1 - \eta)$. Set the distribution $\mathcal{D}' = \frac{1}{2} \text{Rad}(1 - \frac{1}{d})^{\otimes d} + \frac{1}{2} \text{Unif}(\{\pm 1\}^d)$ and denote $\tilde{\mathcal{D}}'$ the distribution of $(y, \mathbf{x})$ with $\mathbf{x} \sim \mathcal{D}'$. Let's compute the correlation of the label with each coordinate x_i , for $i = 1, \dots, d$: using that $\mathbb{E}_{\text{Rad}(1 - \frac{1}{d})}[x_i] = 1 - 2/d$,

$$\mathbb{E}_{\tilde{\mathcal{D}}'}(yx_i) = (1 - 2\eta)\mathbb{E}_{\mathcal{D}'}(\chi_S(\mathbf{x})x_i) = \begin{cases} \frac{1}{2}(1 - 2\eta)(1 - \frac{2}{d})^{|S|+1} & \text{if } i \notin S \\ \frac{1}{2}(1 - 2\eta)(1 - \frac{2}{d})^{|S|-1} & \text{if } i \in S. \end{cases}$$

Then, $\Delta = \frac{1}{2}(1 - 2\eta)(1 - \frac{2}{d})^{|S|-1}(1 - (1 - \frac{2}{d})^2)$ is the difference in the correlation between $i \in S$ and $i \notin S$. Note that $\Delta = \Theta((1 - 2\eta)/d)$.

Let $\hat{T}_i = \frac{1}{m} \sum_{j=1}^m y^j x_i^j$ be the empirical estimate of $\mathbb{E}_{\tilde{\mathcal{D}}'}[yx_i]$. We estimate the support $\hat{S}$ of the parity by taking the set of $i \in [d]$ such that $\hat{T}_i > \frac{1}{2}(\min_i \hat{T}_i + \max_i \hat{T}_i)$. By Hoeffding and union bound,

$$\Pr[\sup_{i \in [d]} |\hat{T}_i - \mathbb{E}_{\tilde{\mathcal{D}}'}(yx_i)| > t] \leq 2d \exp(-2mt^2).$$

Taking $t = \frac{\Delta}{8}$, and $m = \frac{64}{\Delta^2} \log \frac{2d}{\epsilon}$, with probability $1 - \epsilon$ we have that for all i $|\hat{T}_i - \mathbb{E}_{\tilde{\mathcal{D}}'}(yx_i)| < \frac{\Delta}{8}$ so in particular $\hat{T}_i > \frac{1}{2}(\min_i \hat{T}_i + \max_i \hat{T}_i)$ if and only if $i \in S$. Thus we recover the parity function with number of sample $m = O(\frac{d^2 \log \frac{2d}{\epsilon}}{(1 - 2\eta)^2})$. Therefore, for the 0 - 1 loss it holds that it is at most $\epsilon + \eta$ in this case. $\square$

Proof of Theorem 4.5. We consider a parity function $\chi_S(\mathbf{x})$ on the hypercube, with $k := |S| = d/2$. Other k will follow similarly. The goal is to learn this function with a two-layer neural network with activation $\text{ReLu}(x) = (x)_+$.

We consider a two-layer neural network as follow:

$$f_{\text{NN}}(\mathbf{x}; \boldsymbol{\theta}) = \sum_{j \in [N]} a_j \text{ReLu}(\langle \mathbf{w}_j, \mathbf{x} \rangle + b_j),$$

where $\boldsymbol{\theta} = (\mathbf{a}, \mathbf{W}, \mathbf{b}) \in \mathbb{R}^{N(d+2)}$ are the parameters of the network. We will choose as initialization $a_j^0 \in \{\pm 1\}$ and $b_j^0 \in \{-d - 1, -d, \dots, d, d + 1\}$ such that all combinations of (a^0, b^0) are covered. We can either use N sufficiently large (but polynomial in d) or and initialize a_j^0 and b_j^0 iid over these sets but for simplicity we take exactly $(4d + 6)$ neurons, one for each combination (a^0, b^0) . Note that with this choice,

$$f_{\text{NN}}(\mathbf{x}; \boldsymbol{\theta}^0) = 0.$$

We will consider a layerwise training procedure, where we first train the $\mathbf{w}_j$'s with one gradient step with step size s and ℓ_1 regularization with parameter λ_1 , followed by training the a_j 's with (online) SGD with ℓ_2 regularization with parameter λ_2 . Layerwise training is standard in the theory of neural network learning literature (Abbe et al., 2023a; Barak et al., 2022; Bietti et al., 2022; Dandi et al., 2023).

We will use the following two lemmas in our proof. First, we analyze the one step population gradient on the first layer weights.

Lemma C.1 (First Layer Population Gradient Update). *A one step population gradient update on the first layer weights $\mathbf{W}$ with ℓ_1 -regularization λ and gradient step s updates the weights as*

$$\mathbf{w}_j^1 = s \cdot \rho_\lambda((1 - 2\eta)\mathbb{E}_{\mathcal{D}'}[\chi_S(\mathbf{x})\mathbf{x}\mathbb{1}[b_j \geq 0]]),$$

where $\rho(x; \lambda) = \text{sign}(x)(|x| - \lambda)_+$ denotes soft-thresholding and applies component wise.

Proof of Lemma C.1. For the population gradient descent with stepsize s and regularization λ the weight update is

$$\mathbf{w}_j^1 = \text{prox}_{s\lambda\|\cdot\|}(\mathbf{w}_j^0 - s\nabla_{\mathbf{w}_j}\mathcal{L}) = s\rho_\lambda(\mathbf{w}_j^0 - s\nabla_{\mathbf{w}_j}\mathcal{L}).$$

Since we are taking the population gradient, $\mathcal{L} = \mathbb{E}_{\tilde{\mathcal{D}}'}[(\chi_S(\mathbf{x}) - f_{\text{NN}}(\mathbf{x}, \boldsymbol{\theta}))^2]$ so we have that

$$\nabla_{\mathbf{w}_j}\mathcal{L} = (1 - 2\eta)\mathbb{E}_{\mathcal{D}'}[\chi_S(\mathbf{x})\mathbf{x}\mathbb{1}[b_j \geq 0]].$$

We initialized $\mathbf{w}_j^0 = 0$ so this finished the proof of the lemma. $\square$

Second, we recall the following useful upper bound on online SGD on a ridge regularized linear regression with features $\phi(\mathbf{x}) \in \mathbb{R}^p$. Denote the loss

$$\mathcal{L}(\mathbf{a}) = \frac{1}{2}\mathbb{E}_{\mathbf{x}}[(\mathbb{E}[y|\mathbf{x}] - \langle \mathbf{a}, \phi(\mathbf{x}) \rangle)^2], \quad \mathcal{L}_\lambda(\mathbf{a}) = \mathcal{L}(\mathbf{a}) + \frac{\lambda}{2}\|\mathbf{a}\|_2^2.$$

We run online SGD

$$\mathbf{a}^{t+1} = (1 - \lambda s)\mathbf{a}^t + s(y^t - \langle \mathbf{a}^t, \phi(\mathbf{x}^t) \rangle)\phi(\mathbf{x}^t). \quad (1)$$

Lemma C.2 (Online SGD on ridge regression (Abbe et al., 2023a)). *There exists a universal constant $C > 0$ such that the following holds. Suppose there exists $B_y, B_\phi \geq 1$ such that $\|\varphi(\mathbf{x})\|_2 \leq B_\phi$ and $|y| \leq B_y$, then for any $\lambda \leq 1$ and $\mathbf{a}_{\text{cert}} \in \mathbb{R}^p$, we have*

$$\mathcal{L}(\mathbf{a}^t) \leq \mathcal{L}_\lambda(\mathbf{a}_{\text{cert}}) + CB_\phi^2 \left\{ (1 - \lambda s)^{2t} \left(\|\mathbf{a}^0\|_2^2 + \frac{B_y^2}{\lambda} \right) + \log\left(\frac{t}{\delta}\right) \frac{sB_\phi^4 B_y^2}{\lambda^2} \right\}$$

with probability at least $1 - \delta$.

Proof Outline. First, we will show how the parity function can be represented using a 2-layer network of width $4d + 6$. Then we will show that with layerwise training of the first layer, we can select the regularization and stepsize so that one step population gradient picks out the right features. Then, using concentration in $\|\cdot\|_\infty$ norm of the gradient with m samples, we will show that the empirical first layer weights are close to one step population gradient. Finally, we will show that the second layer weights learned by the SGD are close to the ones needed for representing the parity function using the previously established features. Crucially, this will also hold in ℓ_∞ norm, which will allow us to establish the result for any test distribution $\mathcal{D}$.

First Layer Training with Population Gradient. Denote $\mathbf{w}_S$ the vector with 1 on the support S and 0 otherwise, and $g_S : \mathbb{R} \rightarrow \mathbb{R}$ a function such that $\chi_S(\mathbf{x}) = g_S(\langle \mathbf{w}_S, \mathbf{x} \rangle)$ (i.e., $g_S(k - 2j) = (-1)^j$). For simplicity, we assume below that k is odd. Note that we can take

$$\begin{aligned} g_S(x) = & -\text{ReLU}(x + k + 1) + \text{ReLU}(x) + 2 \sum_{j=0}^{(k-1)/2} (-1)^j \cdot \text{ReLU}(x + k - 2j) \\ & + \text{ReLU}(-x + k + 1) - \text{ReLU}(-x) - 2 \sum_{j=0}^{(k-1)/2} (-1)^j \cdot \text{ReLU}(-x + k - 2j). \end{aligned} \quad (2)$$

Let $\mathcal{D}_1 = \text{Rad}(1 - 1/d)^{\otimes d}$, i.e. each coordinate is 1 with probability $1 - 1/d$ and -1 with probability $1/d$. Let $\mathcal{D}_2$ be defined as sampling of $\mathbf{x}$ in the following way: Draw $k \sim \text{Unif}(\{d, d-2, d-4, \dots, -d\})$, and then sample x conditioned on $\mathbf{1}^T \mathbf{x} = k$ uniform on the hypercube (i.e. it is a reweighted distribution on the sliced hypercube, where each slice has equal distribution). Then take

$$\mathcal{D}' = \frac{1}{2}\mathcal{D}_1 + \frac{1}{2}\mathcal{D}_2.$$

Therefore, we have that

$$\mathbb{E}_{\mathcal{D}'}[yx_i] = (1 - 2\eta)\mathbb{E}_{\mathcal{D}'}[\chi_S(\mathbf{x})x_i] = \frac{1 - 2\eta}{2} (\mathbb{E}_{\mathcal{D}_1}[\chi_S(\mathbf{x})x_i] + \mathbb{E}_{\mathcal{D}_2}[\chi_S(\mathbf{x})x_i]).$$

Similarly as above, we have

$$\mathbb{E}_{\mathcal{D}_1}[\chi_S(\mathbf{x})x_i] = \begin{cases} (1 - 2/d)^{|S|+1} & \text{if } i \notin S, \\ (1 - 2/d)^{|S|-1} & \text{if } i \in S. \end{cases}$$

while the expectation with respect to the second distribution yields

$$\mathbb{E}_{\mathcal{D}_2}[\chi_S(\mathbf{x})x_i] = \begin{cases} 0 & \text{if } |S| \text{ is even,} \\ \frac{1}{|S|-1} & \text{if } |S| \text{ odd and } i \in S \\ \frac{1}{|S|+1} & \text{if } |S| \text{ odd and } i \notin S. \end{cases}$$

From Lemma C.1, a one step population gradient ℓ_1 -regularization on the first layer weights with regularization parameter λ_1 gives

$$\mathbf{w}_j^1 = s_1 \cdot \rho_{\lambda_1}((1 - 2\eta)\mathbb{E}_{\mathcal{D}'}[\chi_S(\mathbf{x})\mathbf{x}\mathbb{1}[b_j \geq 0]]).$$

From the computation above we can choose the regularization parameter λ_1 so that λ_1 is in between these values for $x_i \in S$ and $x_i \notin S$:

$$\lambda_1 = \begin{cases} (1 - 2\eta)(1 - 2/d)^{|S|} & \text{if } |S| \text{ is even,} \\ (1 - 2\eta)(1 - 2/d)^{|S|} + \frac{1}{|S|} & \text{if } |S| \text{ odd.} \end{cases}$$

By the choice of λ_1 as above and stepsize s_1 as

$$s_1 = \begin{cases} [(1 - 2\eta)(1 - \frac{2}{d})^{|S|-1}/d]^{-1} & \text{if } |S| \text{ is even} \\ [(1 - 2\eta)((1 - \frac{2}{d})^{|S|-1} + \frac{1}{|S|-1})/d]^{-1} & \text{if } |S| \text{ is odd.} \end{cases}$$

(note that we fix the size the parameter to be $|S| = d/2$), we get the after one step of population gradient descent, we have

$$\bar{\mathbf{w}}_j^1 = a_j^0 \mathbf{w}_S \mathbb{1}[b_j \geq 0].$$

First Layer Empirical Gradient Update. Now, let's consider the empirical gradient update, we have by Hoeffding inequality and union bound, concentration in $\|\cdot\|_\infty$ of the gradient with m samples: it holds with probability at least $1 - \delta$ (where M is the width of the network)

$$\sup_{j \in [M]} \|\mathbf{w}_j^1 - \bar{\mathbf{w}}_j^1\|_\infty \leq \sqrt{\frac{1}{m} \log \frac{dM}{\delta}}.$$

We will denote $\hat{\mathbf{W}}$ the first layer weights after this first (empirical) gradient step, and $\bar{\mathbf{W}}$ the first layer weights after this step with population gradient. In particular, after one population gradient step, we get the $2d + 4$ neurons:

$$\text{ReLU}(\langle \mathbf{w}_S, \mathbf{x} \rangle + b_j), \quad \text{ReLU}(-\langle \mathbf{w}_S, \mathbf{x} \rangle + b_j), \quad b_j \in \{0, 1, 2, \dots, d + 1\}.$$

Second Layer Empirical Gradient Update. Let's run SGD on the second layer weights a_j 's with ridge regularization as in (1). Let's apply Lemma C.2 to our setting. We have $p = 2d + 4$. There exists a universal constant $C > 0$ such that

$$B_y = 1, \quad B_\phi = Cd, \quad \|\mathbf{a}^0\|_2^2 \leq Cd.$$

Consider $\mathbf{a}_{\text{cert}} = (1 - 2\eta)\mathbf{a}^*$ with $\mathbf{a}^*$ as defined in (2): we have $f_{\text{NN}}(\mathbf{x}, \mathbf{a}_{\text{cert}}, \bar{\mathbf{W}}) = (1 - 2\eta)\chi_S(\mathbf{x})$. It follows that

$$\begin{aligned} \|f_{\text{NN}}(\mathbf{a}_{\text{cert}}, \hat{\mathbf{W}}) - (1 - 2\eta)\chi_S\|_{L_2}^2 &= \|f_{\text{NN}}(\mathbf{a}_{\text{cert}}, \hat{\mathbf{W}}) - f_{\text{NN}}(\mathbf{a}_{\text{cert}}, \bar{\mathbf{W}})\|_{L_2}^2 \\ &\leq M^2 \|\mathbf{a}_{\text{cert}}\|_\infty^2 \|\sigma(\langle \bar{\mathbf{w}}_j, \cdot \rangle + b_j) - \sigma(\langle \hat{\mathbf{w}}_j, \cdot \rangle + b_j)\|_\infty^2 \\ &\leq Cd^2 M^2 \sup_{j \in [M]} \|\hat{\mathbf{w}}_j - \bar{\mathbf{w}}_j\|_\infty^2. \end{aligned}$$

Thus we get

$$\mathcal{L}_{\lambda_2}(\mathbf{a}_{\text{cert}}) \leq C \frac{d^2 M^2}{m} \log \frac{dM}{\delta} + C\lambda_2 d.$$

We choose $\lambda_2 = \frac{(1-2\eta)^2\epsilon}{Cd}$, $s_2 = \frac{(1-2\eta)^4\epsilon^2}{Cd^8 \log(d/((1-2\eta)^2\epsilon\delta))}$ and $T = C \frac{d^9 \log^2(d/((1-2\eta)^2\epsilon\delta))}{\epsilon^3}$ such that with probability at least $1 - \delta$,

$$\|(1-2\eta)\chi_S - f_{\text{NN}}(\mathbf{a}^T; \hat{\mathbf{W}})\|_{L^2}^2 + \lambda_2 \|\mathbf{a}^T\|_2^2 \leq (1-2\eta)^2\epsilon,$$

with steps and sample complexity scaling as

$$m, T = O\left(\frac{d^9 \log^2\left(\frac{d}{\epsilon\delta(1-2\eta)^2}\right)}{\epsilon^3(1-2\eta)^6}\right).$$

Final Bound on the Infinity Norm. Let's show that this implies that

$$\|\chi_S - \text{sign}(f_{\text{NN}}(\mathbf{a}^T; \hat{\mathbf{W}}))\|_{L^\infty} \leq \epsilon,$$

so that it implies a test error bounded by $\epsilon + \eta$ for any test distribution. First,

$$\|f_{\text{NN}}(\mathbf{x}, \hat{\mathbf{a}}_T, \hat{\mathbf{W}}) - f_{\text{NN}}(\mathbf{x}, \hat{\mathbf{a}}_T, \overline{\mathbf{W}})\| \leq dM \|\hat{\mathbf{a}}_T\|_\infty \sup_j \|\hat{\mathbf{w}}_j - \overline{\mathbf{w}}_j\|_\infty.$$

Furthermore

$$\begin{aligned} & |f_{\text{NN}}(\mathbf{x}, \hat{\mathbf{a}}_T, \overline{\mathbf{W}}) - (1-2\eta)\chi_S(\mathbf{x})| \\ &= \sum_{k=-|S|}^{k=|S|} \Pr(\langle \mathbf{w}_S, \mathbf{x} \rangle = k) \left| \sum_{j=1}^M \hat{a}_j(k + b_j)_+ - (1-2\eta)\chi_S(k) \right|^2. \end{aligned}$$

Thus,

$$\|f_{\text{NN}}(\hat{\mathbf{a}}_T, \overline{\mathbf{W}}) - (1-2\eta)\chi_S\|_\infty \leq \frac{\|f_{\text{NN}}(\hat{\mathbf{a}}, \mathbf{w}^*) - (1-2\eta)\chi_S\|_{L^2}}{\sqrt{\min_{k=-|S|, \dots, |S|} P(\langle \mathbf{w}_S, \mathbf{x} \rangle = k)}}.$$

Note that $P(\langle \mathbf{w}_S, \mathbf{x} \rangle = k) \geq \mathcal{D}_2(\langle \mathbf{w}_S, \mathbf{x} \rangle = k) = \frac{1}{d} \sum_{i=-d}^d \Pr(\langle \mathbf{w}_S, \mathbf{x} \rangle = k | \langle \mathbf{1}, \mathbf{x} \rangle = i) = \frac{1}{d} \sum_{i=0}^d \Pr(k + 1 \text{ in first } |S| \text{ coordinates} | i \text{ total } + 1 \text{ coordinates}) = \frac{1}{n} \sum_{i=k}^{d-|S|+k} \frac{\binom{|S|}{k} \binom{d-|S|}{i-k}}{\binom{d}{i}} = \frac{1}{d} \frac{d+1}{|S|+1}$. This implies that $P(\langle \mathbf{w}_S, \mathbf{x} \rangle = k) \geq \frac{1}{|S|+1} \geq \frac{1}{d+1}$. Therefore, we have that

$$\|f_{\text{NN}}(\mathbf{x}, \hat{\mathbf{a}}_T, \overline{\mathbf{W}}) - (1-2\eta)\chi_S\|_\infty \leq \sqrt{d+1} \|f_{\text{NN}}(\hat{\mathbf{a}}, \overline{\mathbf{W}}) - \chi_S\|_{L^2}.$$

Note that

$$\|f_{\text{NN}}(\hat{\mathbf{a}}, \overline{\mathbf{W}}) - \chi_S\|_{L^2} \leq \|f_{\text{NN}}(\hat{\mathbf{a}}, \hat{\mathbf{W}}) - \chi_S\|_{L^2} + \|f_{\text{NN}}(\hat{\mathbf{a}}, \hat{\mathbf{W}}) - f_{\text{NN}}(\hat{\mathbf{a}}, \overline{\mathbf{W}})\|_{L^2}.$$

Combining the above bounds, we obtain

$$\begin{aligned} & \|f_{\text{NN}}(\hat{\mathbf{a}}_T, \hat{\mathbf{W}}) - (1-\eta)\chi_S\|_\infty \\ & \leq \sqrt{d+1} \left(\|f_{\text{NN}}(\hat{\mathbf{a}}, \overline{\mathbf{W}}) - f_{\text{NN}}(\hat{\mathbf{a}}, \hat{\mathbf{W}})\|_{L^2} + \|f_{\text{NN}}(\hat{\mathbf{a}}, \overline{\mathbf{W}}) - \chi_S\|_{L^2} \right) \\ & \quad + dM \|\hat{\mathbf{a}}_T\|_\infty \sup_j \|\hat{\mathbf{w}}_j - \mathbf{w}_j^*\|_\infty \\ & \leq \sqrt{d+1} \left(dM \|\hat{\mathbf{a}}_T\|_\infty \sup_j \|\hat{\mathbf{w}}_j - \overline{\mathbf{w}}_j\|_\infty + \|f_{\text{NN}}(\mathbf{a}_{\text{cert}}, \hat{\mathbf{W}}) - f_{\text{NN}}(\mathbf{a}_{\text{cert}}, \overline{\mathbf{W}})\|_{L^2} + \frac{1}{T^c} \right) \\ & \quad + \|\hat{\mathbf{a}}_T\|_\infty \sup_j \|\hat{\mathbf{w}}_j - \overline{\mathbf{w}}_j\|_\infty dM. \end{aligned}$$

Therefore, we have

$$\begin{aligned} \|f_{\text{NN}}(\mathbf{x}, \hat{\mathbf{a}}_T, \hat{\mathbf{w}}) - \chi_S(\mathbf{x})\|_\infty & \leq (1 + 2\sqrt{d+1}) \|\hat{\mathbf{a}}_T\|_\infty \sup_j \|\hat{\mathbf{w}}_j - \mathbf{w}_j^*\|_\infty dM + \frac{\sqrt{d+1}}{T^c} \\ & \leq (1 + 2\sqrt{d+1}) \|\hat{\mathbf{a}}_T\|_\infty dM \sqrt{\frac{1}{m} \log \frac{M}{\delta}} + \frac{\sqrt{d+1}}{T^c}. \end{aligned}$$

We can take $m = \text{poly}(d, \frac{1}{\epsilon}, \log \frac{1}{\delta})$ and $T = \text{poly}(d, \frac{1}{\epsilon})$ (since $M = 4d + 6$) so that with probability at least $1 - \delta$

$$\|f_{NN}(\mathbf{x}, \hat{\mathbf{a}}_T, \hat{\mathbf{w}}) - \chi_S(\mathbf{x})\|_\infty \leq \epsilon.$$

This implies that the 0 – 1 loss is bounded by ϵ with probability δ , so we can take $\delta = \epsilon$ so that in expectation it's bounded by 2ϵ . $\square$

We show that noisy parities are f-PDS learnable with standard analyzable gradient descent on a neural network, namely using layerwise training procedure where we first train the bottom layer while holding the top layer fixed and then train the top layer while holding the bottom layer fixed.

Analyzable SGD C.3 (Layerwise SGD). We perform stochastic gradient descent (SGD) on a two-layer network

$$f_{NN}(\mathbf{x}; \boldsymbol{\theta}) = \sum_{j \in [N]} a_j \text{ReLu}(\langle \mathbf{w}_j, \mathbf{x} \rangle + b_j),$$

where $\boldsymbol{\theta} = (\mathbf{a}, \mathbf{W}, \mathbf{b}) \in \mathbb{R}^{N(d+2)}$ are the parameters of the network. We do layerwise training, where we first train the bottom layer while holding the top layer fixed and then train the top layer while holding the bottom layer fixed, with square loss and with stepsize s . We initialize hyperparameters as $a_j^0 \in \{\pm 1\}$, $\mathbf{w}_j^0 = \mathbf{0}$, and $b_j^0 \in \{-d-1, -d, \dots, d, d+1\}$, $s = 2$, and take $N = (4d + 6)$ neurons so that we get one of each combinations (a^0, b^0) .

Theorem C.4 (Noisy Parities are f-PDS Learnable with Analyzable GD). *The parity class Parity_d with label noise $\eta < \frac{1}{2}$ is f-PDS learnable with Analyzable SGD C.3. Namely, if we train with $\mathcal{D}' = \frac{1}{2} \text{Unif}(\{\pm 1\}^d) + \frac{1}{2} \mathcal{D}_S$, where $\mathcal{D}_S$ is uniform over the hypercube outside the support and on the support, all x_i are +1 with probability $\frac{1}{2}$ and -1 with probability $\frac{1}{2}$, on m samples from $\mathcal{D}'$ using T steps of Analyzable SGD C.3 with $m, T = O\left(\frac{d^3 \log^2(\frac{d}{\epsilon^2(1-2\eta)^2})}{\epsilon^3(1-2\eta)^6}\right)$, we have that $\mathbb{E}_{S' \sim (\mathcal{D}', f)^m} [L_{\mathcal{D}, f}(f_{NN}(\mathbf{x}; \boldsymbol{\theta}^{(T)}))] < \epsilon + \eta$.*

Proof of Theorem C.4. We will define the distribution ν_S as the distribution where we have uniform over the hypercube outside the support, and all x_i on the support equal to +1 with probability 1/2, and -1 with probability 1/2. We consider learning our k -parity using the distribution $\mathcal{D}_S$ given by

$$\mathcal{D}_S = \frac{1}{2} (\nu_0 + \nu_S).$$

We consider a two-layer neural network as follow:

$$f_{NN}(\mathbf{x}; \boldsymbol{\theta}) = \sum_{j \in [N]} a_j \text{ReLu}(\langle \mathbf{w}_j, \mathbf{x} \rangle + b_j),$$

where $\boldsymbol{\theta} = (\mathbf{a}, \mathbf{W}, \mathbf{b}) \in \mathbb{R}^{N(d+2)}$ are the parameters of the network. We will choose as initialization $a_j^0 \sim_{i.i.d.} \text{Unif}(\{\pm 1\})$, $\mathbf{w}_j^0 = \mathbf{0}$, and $b_j^0 \sim_{i.i.d.} \text{Unif}(\{-d-1, -d, \dots, d, d+1\})$. We can either use N sufficiently large (but polynomial in d) or for simplicity we take exactly $(4d + 6)$ neurons, one for each combination (a^0, b^0) . Note that with this choice,

$$f_{NN}(\mathbf{x}; \boldsymbol{\theta}^0) = 0.$$

We will consider a layerwise training procedure, where we first train the $\mathbf{w}_j$'s with one gradient step, followed by training the a_j 's with (online) SGD.

We will use concentration over the gradient at initialization to do one gradient step and noise on the population loss (we will only require $m = O(d \log(d))$ samples to do so). For simplicity, we will simply write the gist of the argument.

After one-gradient step:

$$\mathbf{w}_j^1 = \eta(1 - 2\sigma) a_j^0 \mathbb{E}_{\mathcal{D}_S} [\chi_S(\mathbf{x}) \mathbf{x}] \cdot \mathbb{1}[b_j \geq 0].$$

We have

$$\mathbb{E}_{\mathcal{D}_S} [\chi_S(\mathbf{x}) \mathbf{x}] = \frac{1}{2} \mathbb{E}_{\nu_S} [g_S(\langle \mathbf{w}_S, \mathbf{x} \rangle) \mathbf{x}] = \frac{1}{4} [g_S(k) \mathbf{w}_S - g_S(-k) \mathbf{w}_S] = \frac{1}{2} \mathbf{w}_S.$$

We consider (for simplicity) the step size $\eta = 2$. Hence, for $a_j^0 = +1$ and $b_j^0 \geq 0$, we get $w_j^1 = w_S$, and for $a_j^0 = -1$ and $b_j^0 \geq 0$, we get $w_j^1 = -w_S$. For the rest $w_j^1 = 0$. Hence, after one gradient step, we get the $2d + 4$ neurons:

$$\text{ReLU}(\langle w_S, x \rangle + b_j), \quad \text{ReLU}(-\langle w_S, x \rangle + b_j), \quad b_j \in \{0, 1, 2, \dots, d + 1\}.$$

The rest of the proof follows from a similar argument as the previous proof of Theorem 4.5. $\square$

D. Learning k-juntas

D.1. D-DS-PAC learnability by CSQ

We recall that Correlational Statistical Queries (CSQ) algorithms ((Ben-David et al., 1995; Bshouty & Feldman, 2002)) access the data via queries $\phi : \mathbb{R}^d \rightarrow [-1, 1]$ and return $\mathbb{E}_{x,y}[\phi(x)y]$ up to some error tolerance τ . Here, we give a construction that shows D-DS-PAC learnability of juntas by CSQ algorithms.

Theorem D.1 (Juntas are D-DS-PAC Learnable). *Let Junta_d^k be the class of k -juntas. There exists an input distribution $\mathcal{D}'$ over $\{\pm 1\}^d$ and a CSQ algorithm such that for any $f^* \in \text{Junta}_d^k$, and for any label noise ratio $\eta < 1/2$, after $n = O(dk + 2^k)$ queries on $\mathcal{D}'$ of error tolerance $\tau \leq c \frac{1-2\eta}{2^k \log k}$, for a universal constant $c > 0$, outputs an estimator $\hat{f}$ such that:*

$$\hat{f}(x) = f^*(x), \quad \forall x \in \{\pm 1\}^d. \quad (3)$$

Proof. Let $f^* : \{\pm 1\}^d \rightarrow \{\pm 1\}$ be a k -junta. Fix a label noise ratio $\eta \in [0, 1/2)$. Choose k distinct interpolation nodes $\mu_1, \dots, \mu_k$ defined as:

$$\mu_r = \frac{1}{2} \cos((2r - 1)\pi/(2k)), \quad r = 1, \dots, k. \quad (4)$$

For $r \in [k]$ let $\mathcal{D}_r := \text{Rad}(\frac{1+\mu_r}{2})^{\otimes d}$ be the product distribution on $\{\pm 1\}^d$ with $\mathbb{E}[x_i] = \mu_r$ for all $i \in [d]$. Let $\mathcal{D}' := \frac{1}{k} \sum_{r=1}^k \mathcal{D}_r$.

For $r \in [k]$, let $p_r(x)$ be the density of $\mathcal{D}_r$ and let $p(x) = \frac{1}{k} \sum_{r=1}^k p_r(x)$ be the density of $\mathcal{D}'$. Define $\phi_r(x) = \frac{1}{k} \frac{p_r(x)}{p(x)}$ and note that $|\phi_r(x)| \leq 1$, thus these are valid CSQ queries. Let $\tilde{\mathcal{D}}'$ (resp. $\tilde{\mathcal{D}}_r$) denote the joint distribution over $x \sim \mathcal{D}'$ (resp. $x \sim \mathcal{D}_r$) and $y = f(x)\xi$, with $\xi \sim \text{Rad}(1 - \eta)$. For each coordinate $i \in [d]$, and for each $r \in [k]$, denote:

$$m_r := k \cdot \mathbb{E}_{\tilde{\mathcal{D}}'}[y\phi_r(x)] = \mathbb{E}_{\tilde{\mathcal{D}}_r}[y], \quad v_{i,r} := k \cdot \mathbb{E}_{\tilde{\mathcal{D}}'}[yx_i\phi_r(x)] = \mathbb{E}_{\tilde{\mathcal{D}}_r}[yx_i]. \quad (5)$$

Define the ‘denoising’ function $s_i(r) := v_{i,r} - \mu_r m_r$ and note that

$$s_i(r) = (1 - 2\eta)(1 - \mu_r^2) \cdot \sum_{T:i \in T} \hat{f}(T) \mu_r^{|T|-1} = (1 - 2\eta)(1 - \mu_r^2) P_i(\mu_r), \quad (6)$$

where $\hat{f}(T)$ are the Fourier-Walsh coefficients of f under the standard basis, and where we defined $P_i(\mu_r) := \sum_{T \subseteq [d]: i \in T} \hat{f}(T) \mu_r^{|T|-1}$. Note that $P_i(\mu)$ is a polynomial of degree at most $k - 1$. Furthermore, $P_i(\mu) \neq 0$ if and only if i is in the support of f . Run the following algorithm to identify the support of f :

- For $r \in [k]$ and $i \in [d]$ get m_r and $v_{i,r}$ with $k + dk$ queries, and compute $s_i(r)$.
- For each coordinate i , form evaluations: $\hat{Z}_i(r)/(1 - \mu_r^2)$.
- Interpolate the unique polynomial Q_i of degree $< k$ that passes through $(\mu_r, \hat{Z}_i(r))$.
- Output the support estimate: $\hat{S} = \{i \in [d] : Q_i \text{ is not identically zero}\}$.

Suppose each query is perturbed by at most τ . Then, μ_r and $v_{i,r}$ are each τ -close to their true values, so $s_i(r)$ has error at most 2τ . Dividing by $1 - \mu_r^2 \geq 3/4$ (since $|\mu_r| \leq 1/2$), the error in each evaluation $\hat{Z}_i(r)$ is at most $(8/3)\tau$. Note that $\mu_r, r \in [k]$, are Chebyshev interpolation nodes, thus the polynomial interpolation operator has Lebesgue constant

$\Lambda_k = \theta(\log(k))$ at these nodes. Hence, the reconstructed polynomial Q_i differs from $(1 - 2\eta)P_i$ by at most $C\tau \log(k)$, for a universal constant $C > 0$, that does not depend on d, k or f^* . Because f^* is a k -junta, its nonzero Fourier-Walsh coefficients have magnitude at least 2^{-k} . Therefore, if $C\tau \log(k) < \frac{1}{2}(1 - 2\eta)2^{-k}$, then every relevant coordinate i has some coefficient of Q_i remaining strictly nonzero, while every irrelevant coordinate's coefficients remain below threshold. Thus, $\hat{S} = \text{supp}(f^*)$. Since f^* is fully specified by its restriction to $\{\pm 1\}^{\hat{S}}$, these values can be recovered by an additional $O(2^k)$ CSQ queries $\mathbb{E}_{\tilde{\mathcal{D}}}[1(x_{\hat{S}} = z)y]$, each revealing the sign of $\mathbb{E}[y|x_{\hat{S}} = z]$ for $z \in \{\pm 1\}^{\hat{S}}$. Hence, the algorithm outputs a hypothesis $\hat{f}$ that equals f^* on all inputs. $\square$

D.2. GD on neural networks

Theorem D.2 (Formal statement of Theorem 4.8). *Let $\mathcal{F} := \{f \in \text{Junta}_d^k : \mathbb{E}_{\mathcal{D}}[f(x)] = 0\}$, where $\mathcal{D}$ denotes the uniform distribution over $\{\pm 1\}^d$. Consider a two-layer network with a ReLU activation function: $f_{\text{NN}}(x; \theta) = \sum_{j \in [N]} a_j \sigma(\langle w_j, x \rangle + b_j)$, where $\theta = (a, W, b) \in \mathbb{R}^{N(d+2)}$. We use a layerwise training procedure, where we train w_j 's with one gradient step and a_j 's with SGD with the covariance loss (Def D.3). For any $\epsilon > 0$ and any noise level $\eta < 1/2$, there exist $N = O(\log(1/\epsilon)\epsilon^{-1}(1 - 2\eta)^{-1})$, an initialization, stepsizes, a meta-distribution $\mathcal{M}_{\tilde{\mathcal{D}}}$ over distributions $\tilde{\mathcal{D}}$ over $\{\pm 1\}^d$, $m = \tilde{O}(d \log(1/\epsilon))$, $T = O\left(\frac{1}{\epsilon^2(1-2\eta)^2}\right)$ such that for any $f^* \in \mathcal{F}$, after training the neural network with T steps of gradient descent and m samples from $\tilde{\mathcal{D}}$, we have:*

$$\mathbb{E}_{\tilde{\mathcal{D}} \sim \mathcal{M}_{\tilde{\mathcal{D}}}} \mathbb{P}_{x \sim \mathcal{D}}(f^*(x) \neq f_{\text{NN}}(x; \theta^T)) \leq \eta + C\epsilon^c,$$

for some constants $c, C > 0$ that depend only on k .

D.3. Proof of Theorem D.2

Training distribution. For $\mu \in [-1, 1]^d$, denote:

$$\mathcal{D}_{\mu} := \otimes_{i \in [d]} \text{Rad}\left(\frac{\mu_i + 1}{2}\right), \quad (7)$$

where $\text{Rad}(p)$, $p \in [0, 1]$, denotes the Rademacher distribution with parameter p (i.e. $z \sim \text{Rad}(p)$ if and only if $\mathbb{P}(z = 1) = 1 - \mathbb{P}(z = -1) = p$). Let us denote $\tilde{\mathcal{D}}_{\mu} = \frac{1}{2}\mathcal{D} + \frac{1}{2}\mathcal{D}_{\mu}$. We consider the meta-distribution $\mathcal{M}_{\tilde{\mathcal{D}}}$ such that

$$\mathcal{M}_{\tilde{\mathcal{D}}} = \text{Unif}_{\mu \in [-1/2, 1/2]^d} [\tilde{\mathcal{D}}_{\mu}]. \quad (8)$$

In words, we first sample $\mu \sim \text{Unif}[-1/2, 1/2]^d$, and then we draw the training samples from a mixture of the uniform (target) distribution and the shifted $\mathcal{D}_{\mu}$.

The covariance loss. We use the covariance loss, defined as follows (see also (Abbe et al., 2023c)).

Definition D.3 (Covariance loss). Let $(X, Y) = \{(x^s, y^s)\}_{s \in [m]}$ be a dataset with $x^s \in \mathcal{X}$ and $y^s \in \{\pm 1\}$, and let $\bar{y} = \frac{1}{m} \sum_{s \in [m]} y^s$. Let $\hat{f} : \mathcal{X} \rightarrow \mathbb{R}$ be an estimator. The covariance loss is defined as:

$$L_{\text{cov}}(x, y, \bar{y}, \hat{f}) = (1 - cy\bar{y}) \cdot (1 - y\hat{f}(x))_+, \quad (9)$$

where c is a positive constant such that $c \cdot \bar{y} < 1$.

This choice of loss is particularly convenient because it allows us to get non-zero initial population gradients on weights incident to the coordinates in the support of the target k -junta, and zero initial gradients outside the support, simplifying our construction. For binary classification tasks, a small covariance loss implies a small classification error (see e.g. Proposition 1 in (Abbe et al., 2023c)). In particular, we note that if $\mathbb{E}_{\mathcal{D}}[f(x)] = 0$, then L_{cov} corresponds to the standard hinge loss. In our case, for any training distribution $\tilde{\mathcal{D}}_{\mu} \sim \mathcal{M}_{\tilde{\mathcal{D}}}$, we have $|\mathbb{E}_{\tilde{\mathcal{D}}_{\mu}}[f(x)]| < 1/2$, since we assumed $\mathbb{E}_{\mathcal{D}}[f(x)] = 0$. Therefore, we take $c = 2$. Our experiments in Figure 2 use the more common squared loss, and confirm our theoretical findings.

Lemma D.4. *Let $\{x^s\}_{s \in [m]}$ be i.i.d. inputs from $\mathcal{D}'$, for an input distribution $\mathcal{D}'$. Let $\bar{y} = \frac{1}{B} \sum_{s \in [B]} y^s$. If $B \geq 2C \log(d)/\zeta^2$, with probability at least $1 - 2d^{-C}$,*

$$|\bar{y} - \mathbb{E}_{(\mathcal{D}', f)}[y]| \leq \zeta. \quad (10)$$

Proof. By Hoeffding's inequality,

$$\mathbb{P}(|\bar{y} - \mathbb{E}_{(\mathcal{D}', f)}[y]| \geq \zeta) \leq 2 \exp\left(-\frac{B\zeta^2}{2}\right) \leq 2d^{-C}. \quad (11)$$

□

Setup and algorithm. Without loss of generality, we assume that $S = [k]$, where S denotes the set of relevant coordinates. We assume label noise with parameter η , i.e. for each sample s , $y^s = f(x^s)\xi_s$, where the $\xi_s \sim \text{Rad}(1 - \eta)$ and are independent across samples. We train our network with layer-wise stochastic gradient descent (SGD), defined as follows:

$$\begin{aligned} w_{ij}^{t+1} &= w_{ij}^t - \gamma_t \frac{1}{B} \sum_{s=1}^B \partial_{w_{ij}} L_{\text{cov}}(x^s, y^s, \bar{y}, f_{\text{NN}}(x^s; \theta^t)), \\ a_i^{t+1} &= a_i^t - \xi_t \frac{1}{B} \sum_{s=1}^B \partial_{a_i} L_{\text{cov}}(x^s, y^s, \bar{y}, f_{\text{NN}}(x^s; \theta^t)), \end{aligned}$$

where L_{cov} is the covariance loss, $B \in \mathbb{N}$ is the batch size, and $\gamma_t, \xi_t \in \mathbb{R}_+$ are appropriate learning rates. We set $\gamma_t = \gamma \mathbb{1}(t = 0)$ and $\xi_t = \xi \mathbb{1}(t > 0)$, for $\xi, \gamma \in \mathbb{R}_+$, which means that we train only the first layer for one step, and then only the second layer until convergence. We initialize the first layer weights $w_{ij}^0 = 0$ for all $i \in [N], j \in [d]$, and the second layer weights $a_i^0 = \kappa$, for $i \in [N/2]$ and $a_i^0 = -\kappa$, for $i \in [N/2 + 1, N]$, where $\kappa > 0$ is a constant and where we take N to be even for simplicity. Thus, at initialization $f_{\text{NN}}(x; \theta^0) = 0$ for all $x \in \{\pm 1\}^d$. The biases are initialized to $b_i^0 = \kappa$, for all $i \in [N]$. After the first step of training, we draw $b_i^1 \stackrel{i.i.d.}{\sim} \text{Unif}[-L, L]$, where $L \geq \kappa$.

First layer training. Let us fix our training distribution to be $\tilde{\mathcal{D}}_\mu$, and let us denote by $\tilde{\mathcal{D}}_{\mu, f}$ the joint distribution over the input $x \sim \tilde{\mathcal{D}}_\mu$ and label $y = f(x)\xi$, $\xi \sim \text{Rad}(1 - \eta)$. Let $f(x) = \sum_{S \subseteq [d]} \hat{f}_\mu(S) \chi_{S, \mu}(x)$ be the Fourier-Walsh expansion of f with orthonormal basis elements under $\mathcal{D}_\mu$ (see (O'Donnell, 2014)), where $\chi_{S, \mu}(x) := \prod_{i \in S} \frac{x_i - \mu_i}{\sqrt{1 - \mu_i^2}}$ are the basis elements and $\hat{f}_\mu(S) := \mathbb{E}_{\mathcal{D}_\mu}[f(x) \chi_{S, \mu}(x)]$ are the Fourier-Walsh coefficients of f . We further denote by $\hat{f}(S) := \hat{f}_0(S)$ the coefficients under the uniform distribution. Let us first compute the initial *population gradients* of the first layers' weights, which we denote by $\bar{G}_{w_{ij}^0}$. Given our assumptions on the initialization, we have:

$$\begin{aligned} \bar{G}_{w_{ij}^0} &:= -\mathbb{E}_{\tilde{\mathcal{D}}_{\mu, f}} \left[\partial_{w_{ij}^0} L_{\text{cov}}(x^s, y^s, \bar{y}, f_{\text{NN}}(x^s; \theta^0)) \right] \\ &= \mathbb{E}_{\tilde{\mathcal{D}}_{\mu, f}} [a_i^0 \mathbb{1}(w_i^0 x + b_i^0 > 0) x_j \cdot y] - 2 \mathbb{E}_{\tilde{\mathcal{D}}_\mu} [a_i^0 \mathbb{1}(w_i^0 x + b_i^0 > 0) x_j] \cdot \bar{y} \\ &\stackrel{(a)}{=} (1 - 2\eta) \cdot \left(\mathbb{E}_{\tilde{\mathcal{D}}_\mu} [a_i^0 \mathbb{1}(w_i^0 x + b_i^0 > 0) x_j \cdot f(x)] - 2 \mathbb{E}_{\tilde{\mathcal{D}}_\mu} [a_i^0 \mathbb{1}(w_i^0 x + b_i^0 > 0) x_j] \cdot \mathbb{E}_{\tilde{\mathcal{D}}_\mu}[f(x)] \right) + r \\ &\stackrel{(b)}{=} a_i^0 (1 - 2\eta) \cdot \left(\mathbb{E}_{\tilde{\mathcal{D}}_\mu} [x_j f(x)] - 2 \mathbb{E}_{\tilde{\mathcal{D}}_\mu} [x_j] \mathbb{E}_{\tilde{\mathcal{D}}_\mu}[f(x)] \right) + r \\ &\stackrel{(c)}{=} a_i^0 (1 - 2\eta) \cdot \left(\frac{1}{2} \hat{f}(\{j\}) + \frac{1}{2} \mathbb{E}_{\mathcal{D}_\mu} [x_j f(x)] - \frac{1}{2} \mathbb{E}_{\mathcal{D}_\mu} [x_j] \mathbb{E}_{\mathcal{D}_\mu}[f(x)] \right) + r \\ &= \frac{1}{2} a_i^0 (1 - 2\eta) (\hat{f}(\{j\}) + \mathbb{E}_{\mathcal{D}_\mu} [f(x)(x_j - \mu_j)]) + r \\ &\stackrel{(d)}{=} \frac{1}{2} a_i^0 (1 - 2\eta) (\hat{f}(\{j\}) + \hat{f}_\mu(\{j\}) \sqrt{1 - \mu_j^2}) + r \\ &\stackrel{(e)}{=} \alpha_j a_i^0 + r \end{aligned}$$

where: in (a) we apply Lemma D.4 to $\bar{y} = (1 - 2\eta) \mathbb{E}_{\tilde{\mathcal{D}}_\mu}[f(x)] + \delta$, with $|\delta| \leq \zeta$, which leads to $|r| \leq 2\kappa\zeta$, (b) holds because of the initialization that we have chosen, in (c) and (d) we used the definitions of the Fourier coefficients of f , and in (e) we defined $\alpha_j := \frac{1}{2} (1 - 2\eta) (\hat{f}(\{j\}) + \hat{f}_\mu(\{j\}) \sqrt{1 - \mu_j^2})$. The following lemma bounds the discrepancy between the effective gradients, estimated through B samples, and the population gradients.

Lemma D.5. Let $G_{w_{ij}^0} := -\frac{1}{B} \sum_{s=1}^B \partial_{w_{ij}^0} L_{\text{cov}}(x^s, y^s, \bar{y}, f_{\text{NN}}(x^s; \theta^0))$ and let $\bar{G}_{w_{ij}^0} := a_i^0 (\mathbb{E}[x_j y] - 2\mathbb{E}[x_j] \mathbb{E}[y])$, where expectations are under $\tilde{\mathcal{D}}_{\mu, f}$. Assume $|a_i^0| \leq \kappa$. For any $\zeta, \varepsilon > 0$, if $B \geq \frac{50\kappa^2}{\zeta^2} \log\left(\frac{2d+1}{\varepsilon}\right)$, then, with probability at least $1 - 2\varepsilon$,

$$|G_{w_{ij}^0} - \bar{G}_{w_{ij}^0}| \leq \zeta$$

simultaneously for all $i \in [N]$ and $j \in [d]$.

Proof. At initialization,

$$G_{w_{ij}^0} = a_i^0 (\hat{A}_j - 2\hat{B}_j \hat{C}),$$

where

$$\hat{A}_j = \frac{1}{B} \sum_{s=1}^B x_j^s y^s, \quad \hat{B}_j = \frac{1}{B} \sum_{s=1}^B x_j^s, \quad \hat{C} = \frac{1}{B} \sum_{s=1}^B y^s.$$

Let

$$A_j = \mathbb{E}[x_j y], \quad B_j = \mathbb{E}[x_j], \quad C = \mathbb{E}[y].$$

By Hoeffding and a union bound, with probability at least $1 - 2\varepsilon$,

$$|\hat{A}_j - A_j| \leq \delta, \quad |\hat{B}_j - B_j| \leq \delta, \quad |\hat{C} - C| \leq \delta$$

for all $j \in [d]$, provided

$$B \geq \frac{2}{\delta^2} \log\left(\frac{2d+1}{\varepsilon}\right).$$

On this event,

$$\begin{aligned} |G_{w_{ij}^0} - \bar{G}_{w_{ij}^0}| &\leq |a_i^0| (|\hat{A}_j - A_j| + 2|\hat{B}_j \hat{C} - B_j C|) \\ &\leq \kappa(\delta + 4\delta) = 5\kappa\delta, \end{aligned}$$

since $|\hat{B}_j|, |C| \leq 1$. Taking $\delta = \frac{\zeta}{5\kappa}$ gives the claim. $\square$

We make use of the following Lemma, whose proof can be found in ((Cornacchia et al., 2025)[Lemma 10]), which guarantees enough diversity among the hidden features after the first step.

Lemma D.6 ((Cornacchia et al., 2025), Lemma 10). Let $\alpha_j = \frac{(1-2\eta)}{2} (\hat{f}(\{j\}) + \hat{f}_\mu(\{j\}) \sqrt{1 - \mu_j^2})$. For $\epsilon > 0$, there exists a constant $C > 0$ such that, with probability $1 - O(\epsilon^{\frac{1}{k+1}})$ over μ , for all $s, t \in \{\pm 1\}^k$, such that $s \neq t$, $\left| \sum_{j=1}^k \alpha_j (s_j - t_j) \right| \geq C(1 - 2\eta)\epsilon$.

Second layer training. We show that the previous lemmas imply that there exists an assignment of the second layer's weights that achieves small error.

Lemma D.7. Let $\alpha_j = \frac{(1-2\eta)}{2} (\hat{f}(\{j\}) + \hat{f}_\mu(\{j\}) \sqrt{1 - \mu_j^2})$, for $j \in [k]$. Assume $\kappa, \gamma = \theta(1)$. Assume that $b_i \sim \text{Unif}[-L, L]$, with L being a sufficiently large constant depending on k, γ, κ . For $\varepsilon_1, \varepsilon_2 > 0$, if the number of hidden neurons $N > \Omega(L \log(1/\varepsilon_2) \varepsilon_1^{-1} (1 - 2\eta)^{-1})$, with probability $1 - O(\varepsilon_1^{\frac{1}{k+1}} + \varepsilon_2)$, there exists a set of hidden neurons $\{i\}_{i \in [2^k]}$ and a vector $a^* \in \mathbb{R}^{2^k}$ with $\|a^*\|_\infty \leq O(\varepsilon_1^{-1} (1 - 2\eta)^{-1})$ such that for all $x \in \{\pm 1\}^d$,

$$f(x) = \sum_{i=1}^{2^k} a_i^* \text{ReLU} \left(\gamma a_i^0 \sum_{j=1}^k \alpha_j x_j + b_i \right). \quad (12)$$

Proof. Let us consider only hidden neurons with $a_i^0 = \kappa$. For all $s \in \{\pm 1\}^k$, let $v_s := \gamma \kappa \sum_{j=1}^k \alpha_j s_j$, and let us order the $(v_{s_l})_{l \in [2^k]}$ in increasing order, i.e. such that $v_{s_l} < v_{s_{l+1}}$ for all $l \in [2^k - 1]$. For simplicity, we denote $v_l = v_{s_l}$. By Lemma D.6, we have that with probability $1 - O(\varepsilon_1^{\frac{1}{k+1}})$ over μ , $\min_{l \in [2^k - 1]} v_{l+1} - v_l > C\gamma\kappa\varepsilon_1(1 - 2\eta)$, for some constant

$C > 0$. If $N > \Omega(L \log(1/\varepsilon_2) \varepsilon_1^{-1} (1 - 2\eta)^{-1})$, then with probability $1 - O(\varepsilon_2)$ there exists a set of 2^k hidden neurons $(b_l)_{l \in [2^k]}$ such that for all $l \in [2^k]$, $b_l \in (-v_l + \frac{\Delta}{4}, -v_l + \frac{\Delta}{2})$, where $\Delta := \gamma \kappa (1 - 2\eta) \varepsilon_1$ and for simplicity we let $v_0 = -L$. Let us define the matrix $M \in \mathbb{R}^{2^k \times 2^k}$, with entries:

$$M_{n,m} = \text{ReLU}(v_n + b_m), \quad n, m \in [2^k].$$

Then, by construction, M is lower triangular, i.e. $M_{n,m} = 0$ if $m \geq n + 1$. Furthermore, by the construction above, the diagonal entries of M are non-zero. Thus, M is invertible. Let us denote by $F \in \mathbb{R}^{2^k}$ the vector such that for all $l \in [2^k]$, the l -th entry is given by $F_l = f(s_l)$. Then, $a^* = M^{-1}F$ and

$$\begin{aligned} \|a^*\|_\infty &\leq \|M^{-1}\|_\infty \|F\|_\infty \\ &\leq C \cdot \frac{1}{\gamma \kappa \varepsilon_1 (1 - 2\eta)}, \end{aligned}$$

for a constant $C > 0$. □

By combining the lemmas above, we obtain that for $\kappa, \gamma, L = \theta(1)$, $B \geq \Omega(d \log(d)^2 \log(Nd/\varepsilon))$, $N \geq \Omega(\log(1/\varepsilon) \varepsilon^{-1} (1 - 2\eta)^{-1})$ with probability $1 - O(\varepsilon^c)$, for some $c > 0$, there exists a set of 2^k hidden neurons $\{i\}_{i \in [2^k]}$ such that

$$\begin{aligned} \forall j \in [k], i \in [2^k] : |w_{ij}^1 - \gamma \kappa \alpha_j| &< \frac{1}{\sqrt{d} \log(d)}; \\ \forall j \notin [k], i \in [2^k] : |w_{ij}^1| &< \frac{1}{\sqrt{d} \log(d)}. \end{aligned}$$

Moreover, their biases b_i satisfy the interpolation property of Lemma D.7. By a slight abuse of notation, let us denote by $a^* \in \mathbb{R}^N$ the N -dimensional vector whose entries corresponding to the hidden neurons $\{i\}_{i \in [2^k]}$ are given by Lemma D.7, and the other entries are zero. For all $i \in [N]$, let $w_i^* \in \mathbb{R}^d$ be such that $w_i^* = \gamma \kappa \alpha_j \mathbb{1}(j \in [k])$. Let $\hat{\theta} = (a_i^*, w_i^1, b_i)_{i \in [N]}$ and $\theta^* = (a_i^*, w_i^*, b_i)_{i \in [N]}$. Note that, for all $(x, y) \sim \tilde{\mathcal{D}}_{\mu, f}$ we have:

$$\left| \text{ReLU}(w_i^1 x + b_i) - \text{ReLU}(w_i^* x + b_i) \right| \leq \left| \sum_{j=1}^k (w_{ij}^1 - \gamma \kappa \alpha_j) x_j \right| + \left| \sum_{j=k+1}^d w_{ij}^1 x_j \right| \quad (13)$$

$$\leq \frac{2k}{\sqrt{d} \log(d)} + \left| \sum_{j=k+1}^d w_{ij}^1 x_j \right|. \quad (14)$$

Now, $\sum_{j=k+1}^d w_{ij}^1 x_j = \sum_{j=k+1}^d w_{ij}^1 \mu_j + \sum_{j=k+1}^d w_{ij}^1 (x_j - \mu_j) := m_i + Z_i$. Then,

$$\mathbb{E} Z_i^2 = \sum_{j=k+1}^d (w_{ij}^1)^2 \text{Var}(x_j) = O(1/\log(d)^2). \quad (15)$$

On the other hand, since μ_j are i.i.d. in $[-1/2, 1/2]$, m_i is sub-Gaussian with $\text{Var}(m_i) \leq 1/\log(d)^2$. Hence,

$$\mathbb{P}(|m_i| > t) \leq 2 \exp(-t^2 \log^2(d)/2). \quad (16)$$

Setting $t = O(1/\sqrt{\log(d)})$, we get that with probability $1 - 1/d$ over μ , $|m_i| < 1/\sqrt{\log(d)}$. Let $\mathcal{E}$ denote such event. Combining terms,

$$\mathbb{E}_{\tilde{\mathcal{D}}_\mu} \left[\left| \sum_{j=k+1}^d w_{ij}^1 x_j \right| \middle| \mathcal{E} \right] = O(1/\sqrt{\log(d)}). \quad (17)$$

Thus,

$$\mathbb{E}_{\tilde{\mathcal{D}}_{\mu,f}} [L_{\text{cov}}(x, y, \bar{y}, f_{\text{NN}}(x; \hat{\theta}))] - \mathbb{E}_{\tilde{\mathcal{D}}_{\mu,f}} [L_{\text{cov}}(x, y, \bar{y}, f(x))] \quad (18)$$

$$\stackrel{(a)}{\leq} 2 \cdot \mathbb{E}_{\tilde{\mathcal{D}}_{\mu}} |f_{\text{NN}}(x; \hat{\theta}) - f(x)| \quad (19)$$

$$\stackrel{(b)}{=} 2 \cdot \mathbb{E}_{\tilde{\mathcal{D}}_{\mu}} |f_{\text{NN}}(x; \hat{\theta}) - f_{\text{NN}}(x; \theta^*)| \quad (20)$$

$$\leq 2 \sum_{i=1}^N |a_i^*| \mathbb{E}_{\tilde{\mathcal{D}}_{\mu}} |\text{ReLU}(w_i^1 x + b_i) - \text{ReLU}(w_i^* x + b_i)| \quad (21)$$

$$\leq 2 \|a^*\|_{\infty} \sum_{i: |a_i^*| \neq 0} \mathbb{E}_{\tilde{\mathcal{D}}_{\mu}} |\text{ReLU}(w_i^1 x + b_i) - \text{ReLU}(w_i^* x + b_i)| \quad (22)$$

$$= O\left(\frac{\varepsilon^{-1}(1-2\eta)^{-1}}{\sqrt{\log d}}\right), \quad (23)$$

where (a) uses that $(1 - c\bar{y}y) \leq 1 + c|\bar{y}| \leq 2$, and

$$|(1 - yf_{\text{NN}}(x; \hat{\theta}))_+ - (1 - yf(x))_+| \leq |y| \cdot |f_{\text{NN}}(x; \hat{\theta}) - f(x)| = |f_{\text{NN}}(x; \hat{\theta}) - f(x)|,$$

and (b) uses Lemma D.7.

We then use the following well-known result on the convergence of SGD on convex losses, to show that training only the second layer with a convex loss achieves small error.

Theorem D.8 ((Shalev-Shwartz & Ben-David, 2014)). *Let $\mathcal{L}$ be a convex function and let $a^* \in \arg \min_{\|a\|_2 \leq \mathcal{B}} \mathcal{L}(a)$, for some $\mathcal{B} > 0$. For all t , let α^t be such that $\mathbb{E}[\alpha^t | a^t] = -\nabla_{a^t} \mathcal{L}(a^t)$ and assume $\|\alpha^t\|_2 \leq A$ for some $A > 0$. If $a^{(0)} = 0$ and for all $t \in [T]$ $a^{t+1} = a^t + \gamma \alpha^t$, with $\gamma = \frac{\mathcal{B}}{A\sqrt{T}}$, then*

$$\frac{1}{T} \sum_{t=1}^T \mathcal{L}(a^t) \leq \mathcal{L}(a^*) + \frac{\mathcal{B}A}{\sqrt{T}}.$$

We choose $\mathcal{B} = \Omega(\varepsilon^{-1}(1-2\eta)^{-1})$. In our case we have $\|\alpha^t\|_2 \leq C\sqrt{N}$, for some constant $C > 0$.

We then apply the calibration inequality for hinge-type losses under symmetric label noise. For a predictor $\hat{f}$, a direct case analysis on $\text{sign}(f\hat{f})$ and $|f\hat{f}|$ gives the conditional bound

$$\mathbb{E}_{\xi}[(1 - 2y\bar{y})((1 - y\hat{f})_+ - (1 - yf)_+) | x] \geq ((1 - 2\eta) - 2|\bar{y}|) \cdot \mathbf{1}(\text{sign}(\hat{f}(x)) \neq f(x)),$$

so that, after taking expectation over x ,

$$\mathbb{P}_{\tilde{\mathcal{D}}_{\mu}}(\text{sign}(\hat{f}(x)) \neq f(x)) \leq \frac{\mathbb{E}_{\tilde{\mathcal{D}}_{\mu,f}}[L_{\text{cov}}(\hat{f})] - \mathbb{E}_{\tilde{\mathcal{D}}_{\mu,f}}[L_{\text{cov}}(f)]}{(1 - 2\eta) - 2|\bar{y}|}, \quad (24)$$

Since $\mathbb{E}_{\mathcal{D}}[f] = 0$, we have $|\mathbb{E}_{\tilde{\mathcal{D}}_{\mu}}[f(x)]| \leq \frac{1}{2} - 2^{-2k}$ and hence

$$(1 - 2\eta) - 2|\mathbb{E}[y]| = (1 - 2\eta)(1 - |\mathbb{E}_{\mathcal{D}_{\mu}}[f]|) \geq (1 - 2\eta)2^{-2k},$$

and by concentration (Lemma D.4)

$$(1 - 2\eta) - 2|\bar{y}| \geq (1 - 2\eta)2^{-2k} + o_d(1).$$

Thus, there exists a constant $\rho_k > 0$ depending only on k such that with probability $1 - O(\epsilon^c)$ over μ , the denominator of (24) is at least $\rho_k(1 - 2\eta)$. Combining with (23), with the convergence guarantee of Theorem D.8 applied to the convex

excess risk (which gives optimization error $O(\epsilon^c)$ after $T = \Omega(1/(\epsilon^2(1 - 2\eta)^2))$ steps), and transferring the bound from $\tilde{\mathcal{D}}_\mu = \frac{1}{2}\mathcal{D} + \frac{1}{2}\mathcal{D}_\mu$ to $\mathcal{D}$ at the cost of a factor of two, we get

$$\mathbb{P}_{\mathcal{D}}(\text{sign}(f_{\text{NN}}(x; \theta^T)) \neq f(x)) \leq \frac{C\epsilon^c}{1 - 2\eta}$$

on the high-probability event of Lemmas D.6–D.7 together with the μ -event above, which jointly hold with probability $1 - O(\epsilon^c)$. For noisy test labels $y = f(x)\xi$, with $\xi \sim \text{Rad}(1 - \eta)$, we have

$$L_{\mathcal{D},f}^\eta(\hat{h}) := \Pr_{x \sim \mathcal{D}, \xi} [\hat{h}(x) \neq f(x)\xi] = \eta + (1 - 2\eta) \Pr_{x \sim \mathcal{D}} [\hat{h}(x) \neq f(x)].$$

Therefore, using the clean-error bound above, and taking expectation over $\tilde{\mathcal{D}} \sim \mathcal{M}_{\tilde{\mathcal{D}}}$ gives:

$$\mathbb{E}_{\tilde{\mathcal{D}} \sim \mathcal{M}_{\tilde{\mathcal{D}}}} L_{\mathcal{D},f}^\eta(\text{sign}(f_{\text{NN}}(\cdot; \theta^T))) \leq \eta + C\epsilon^c.$$

E. DS-PAC and Membership Queries: Proofs

Proof of Theorem 5.2. Assume we have a D-DS-PAC learning algorithm A for $\mathcal{H}$, where the algorithm draws its training data S' from the training distribution $\mathcal{D}'$. By definition, the expected excess error is small: $\mathbb{E}_{S' \sim (\mathcal{D}', f)^{m(\epsilon)}} [L_{\mathcal{D},f}(A(S')) - L_{\mathcal{D},f}(\mathcal{H})] \leq \epsilon$. Then an NA-MQ algorithm can simply query those sample points S' and simulate the algorithm A .

Moreover, learnability in the realizable setting in D-DS-PAC also implies learnability in the presence of label noise: after sampling a training set, we can resample each query sufficiently many times so that, by the coupon collector argument, every queried example is observed multiple times, and the true label can be recovered by majority vote. $\square$

Proof of Theorem 5.3. Let A be the given NA-MQ learner. Run A only to generate its query list $Q = (x_1, \dots, x_m)$ and $m \leq m_0(\epsilon/2)$. Let $U = \text{supp}(Q)$, so $|U| \leq m$. Define the training distribution D'_Q to be the uniform distribution over U . Let $\mathcal{M}_{\mathcal{D}}$ be the distribution over training distributions induced by running A (which can be randomized) to generate its query list Q and setting D'_Q uniform over Q .

Choose failure probability $\delta = \epsilon/2$. By the coupon-collector bound, if $\tilde{m} \geq \Omega(|U|(\log |U| + \log(1/\delta)))$, then with probability at least $1 - \delta$ every $u \in U$ appears in S . On this event, the learner reconstructs the full labeled set $\{(u, f(u)) : u \in U\}$ (taking the first occurrence for each u). It then runs A on this set to obtain $\hat{h}$.

Conditioned on full coverage of U , the distribution of the reconstructed labeled query list is exactly the NA-MQ transcript that A expects. Thus

$$\mathbb{E} [L_{\mathcal{D},f}(\hat{h}) | \text{coverage}] \leq \inf_{h \in \mathcal{H}} L_{\mathcal{D},f}(h) + \epsilon/2.$$

On the failure event (probability $\leq \delta$), the learner outputs a fixed $h_0 \in \mathcal{H}$, which adds at most δ to the error. Therefore,

$$\mathbb{E}[L_{\mathcal{D},f}(\hat{h})] \leq \inf_{h \in \mathcal{H}} L_{\mathcal{D},f}(h) + \epsilon/2 + \delta \leq \inf_{h \in \mathcal{H}} L_{\mathcal{D},f}(h) + \epsilon.$$

Using $|U| \leq m_0(\epsilon/2)$ and $\delta = \epsilon/2$, we have

$$\tilde{m} = O\left(m_0(\epsilon/2) (\log m_0(\epsilon/2) + \log(1/\epsilon))\right).$$

This gives the RDSPAC sample bound. The procedure is polynomial-time assuming A is.

Deterministic case. If A is deterministic, then Q (and hence U) is fixed given $\mathcal{D}$, so the training distribution can be chosen deterministically as $D' := D'_Q$. This yields DDSPAC learnability with the same sample bound. $\square$

F. Classifying Hypothesis Classes Across Frameworks

In this section, we elaborate on the classification of hypothesis classes within the PDS framework and related learning models.

- **Parities:** For $a \in \{0, 1\}^d$, define the parity $\chi_a(x) = \bigoplus_{i=1}^d a_i x_i = \langle a, x \rangle \bmod 2$. The parity class over d bits is defined as $\text{Parity}_d = \{\chi_a : a \in \{0, 1\}^d\}$. We consider also k -sparse parities, $\text{Parity}_d^k = \{\chi_a : a \in \{0, 1\}^d, \|a\|_0 = k\}$.
- **Juntas:** For $k \leq d$, the k -junta class is $\text{Junta}_d^k = \{f : \{0, 1\}^d \rightarrow \{0, 1\} : \exists S \subseteq [d], |S| \leq k, \exists g : \{0, 1\}^S \rightarrow \{0, 1\} \text{ s.t. } f(x) = g(x_S)\}$.
- **Boolean circuits:** Let B be the set of gates $\{\text{AND}, \text{OR}, \text{NOT}\}$. A Boolean circuit C on d inputs is a finite directed acyclic graph (DAG) with input nodes $x_1, \dots, x_d$, internal nodes labeled by gates in B , and one output node. For $x \in \{0, 1\}^d$, values propagate along edges to define $f_C(x) \in \{0, 1\}$. The size of the circuit is the number of gate nodes. Let $\text{Circuit}_d^{t,s}$ be the set of all Boolean functions on d variables that are computable by Boolean circuits of depth at most t and size at most s .
- **Disjunctive normal form (DNF):** A term is a conjunction of literals. A DNF with m terms is a function $f(x) = T_1(x) \vee \dots \vee T_m(x)$ where each T_j is a term over $\{x_1, \dots, x_d\}$. The class of DNF formulas with at most $s(d)$ terms over d variables is $\text{DNF}_d^s = \{f : \{0, 1\}^d \rightarrow \{0, 1\} : \exists m \leq s(d) \text{ s.t. } f(x) = \bigvee_{j=1}^m T_j(x)\}$.
- **Decision Tree:** A (binary) decision tree on variables $x_1, \dots, x_d$ is a rooted tree whose internal nodes are labeled by variables and whose edges correspond to outcomes $x_i = 0$ or $x_i = 1$, where leaves are labeled by outputs in $\{0, 1\}$. It computes the function given by evaluating the tested variables along the unique root-to-leaf path. Let $\text{DT}_d^{t,s}$ be the set of all Boolean functions on d variables that are computed by (binary) decision trees of depth at most t and size at most s .
- **Sparse functions:** Let $f : \{0, 1\}^d \rightarrow \{-1, 1\}$ be a Boolean function with Fourier expansion $f(x) = \sum_{S \subseteq [d]} \hat{f}(S) \chi_S(x)$, where $\chi_S(x) = (-1)^{\sum_{i \in S} x_i}$ are the parity functions. We say that f is s -sparse if the number of nonzero Fourier coefficients is at most s , i.e., $|\{S \subseteq [d] : \hat{f}(S) \neq 0\}| \leq s$.
- **Deterministic finite automaton (DFA):** Let $\Sigma = \{0, 1\}$. A DFA is a tuple $M = (Q, \Sigma, \delta, q_0, F)$ with finite state set Q , transition function $\delta : Q \times \Sigma \rightarrow Q$, start state $q_0 \in Q$, and accept set $F \subseteq Q$. For length d , M induces a Boolean function $f_M : \{0, 1\}^d \rightarrow \{0, 1\}$ by $f_M(x_1, \dots, x_d) = \mathbb{I}\{\delta^{(d)}(q_0, x_1, \dots, x_d) \in F\}$, where $\delta^{(d)}$ extends δ to strings. Let DFA_d^s be the set of all Boolean functions on d variables that are computed by a DFA with at most s states.

Juntas. $\log(d)$ -Juntas are known to be deterministically NA-MQ learnable in the realizable setting (Bshouty & Costa, 2016). Because deterministic NA-MQ learners fix their queries in advance, random classification noise can be overcome by repeating each query sufficiently many times (ensuring, by the coupon collector argument, that every queried example is observed multiple times) and taking the majority label. This in turn implies that noisy juntas are also D-DS-PAC learnable. We provide a direct proof of this fact by a different method (using correlation statistical queries), presenting a simple Correlational Statistical Query (CSQ) algorithm for D-DS-PAC learning juntas with random label noise.

In the agnostic setting, this class is learnable in the random walk model (Arpe & Mossel, 2008), and hence also in NA-MQ. Since we established the equivalence between R-DS-PAC and NA-MQ, it follows that juntas are R-DS-PAC learnable in the agnostic setting as well.

DNFs. Feldman (2007) showed that DNFs are learnable under the uniform distribution in the realizable case using non-adaptive membership queries. Consequently, DNFs are also R-DS-PAC learnable, including in the presence of random classification noise. It remains an open question whether this class is D-DS-PAC learnable. Bshouty et al. (2005) further established that DNFs are learnable in the random walk model, which is a more “passive” setting than NA-MQ, while Bartlett et al. (1994) had earlier shown the result for 2-term DNFs. Prior to these works, DNFs were known to be A-MQ learnable under the uniform distribution (Blum et al., 1994; Jackson, 1997). Most recently, Alman et al. (2025) gave a quasi-polynomial time algorithm for DNFs using (adaptive) membership queries that applies to arbitrary distributions.

Decision trees. Bshouty (2018) showed that decision trees of logarithmic depth in the dimension are distribution-free learnable in the realizable case using non-adaptive membership queries. Consequently, they are also R-DS-PAC learnable, including in the presence of random classification noise. It remains an open question whether this class is D-DS-PAC learnable. Bshouty et al. (2005) further established that decision trees are learnable in the random walk model. This class has long been known to be learnable using adaptive membership queries (Kushilevitz & Mansour, 1993), with subsequent improvements by Bshouty & Haddad-Zaknoon (2019). In the agnostic setting, decision trees are learnable under the uniform distribution in A-MQ (Gopalan et al., 2008).

Sparse functions. (Kushilevitz & Mansour, 1993) showed that for all functions $f : \{0, 1\}^n \rightarrow \{0, 1\}$ that are ϵ approximated by a t sparse function in L_2 , there exists a randomized polynomial time algorithm using (adaptive) membership queries that on input f and δ returns h such that with probability $1 - \delta$, h $O(\epsilon)$ approximates f in L_2 . This implies that sparse functions are A-MQ learnable for the uniform distribution.

DFAs. A seminal result by Angluin (1987) showed that this class can be learned using adaptive membership queries. Whether it is also NA-MQ learnable, or whether a separation exists, remains an open question (to the best of our knowledge).

Circuits. Learning general circuits remains computationally intractable even with membership queries. For instance, it is known that constant-depth Boolean circuits (AC^0) and threshold circuits (TC^0) are hard to learn under the uniform distribution, even in the A-MQ model (Kharitonov, 1993). While the precise hardness assumptions for Boolean circuits are not always considered “standard”, threshold circuits already provide strong evidence of intractability: depth-4 TC circuits can implement pseudorandom functions (Krause & Lucks, 2001; Naor & Reingold, 2004), which implies hardness of learning with membership queries for every non-trivial input distribution. Moreover, recent work (Chen et al., 2022, Section 6) shows that such pseudorandom function constructions also yield hardness results for learning real-valued neural networks of depth 5 or 6 under natural distributions such as the Gaussian. Together, these results highlight that circuit classes capable of expressing pseudorandom functions are not learnable, even with powerful query access, unless one is willing to break widely believed cryptographic assumptions.

G. Experiment Details and Additional Experiments

G.1. Experiment Details

All experiments were performed using the PyTorch framework ((Paszke et al., 2019)) and they were executed on NVIDIA Volta V100 GPUs. Each experiment is repeated 5 times and we plot the mean; shaded regions denote ± 1 standard deviation.

Architecture. For the results presented in the main, we used a 2-layer MLP architecture trained by SGD with the square loss. In this Section, we also present some experiments obtained with a 4-layer MLP trained by SGD with the squared loss.

- **2-layer MLP.** This is a fully-connected architecture, with 1 hidden layer of 1024 neurons, and ReLU activation.
- **4-layer MLP.** This is a fully-connected architecture of 3 hidden layers of neurons of size 512, 512, 64, and ReLU activation.

We use PyTorch’s default initialization, which initializes weights of each layer with $\text{Unif}[\frac{1}{\sqrt{\text{dim}_{\text{in}}}}, -\frac{1}{\sqrt{\text{dim}_{\text{in}}}}]$, where dim_{in} is the input dimension of the corresponding layer.

Training procedure. We consider the ℓ_2 loss: $L_{\ell_2}(\hat{y}, y) := (\hat{y} - y)^2$. We sample fresh batches of samples at each iterations. We stop training either when the training loss is less than 0.01, or when 10^6 iterations are performed. We compare PDS with no-PDS, where for PDS is defined as follows for parities and juntas:

- **PDS for Parities:** We select samples from $\mathcal{D}' = \frac{1}{2}\text{Unif}\{\pm 1\}^d + \frac{1}{2}\text{Rad}(1 - 1/d)^{\otimes d}$. The test error is computed on the uniform distribution.
- **PDS for Juntas:** For each experiment, we independently draw $\mu \sim \text{Unif}[-1, 1]^{\otimes d}$. We select training samples from $\mathcal{D}' = \frac{1}{2}\text{Unif}\{\pm 1\}^d + \frac{1}{2} \otimes_{i \in [d]} \text{Rad}((\mu_i + 1)/2)$. The test error is computed on the uniform distribution. The test error is computed on the uniform distribution.

In the **no-PDS** experiments, we select both training and test samples from $\text{Unif}\{\pm 1\}^d$, for both parities and juntas. In all experiments, the test-set is of size 8192.

Hyperparameter tuning. The primary goal of our experiments is to conduct a fair comparison between PDS and no-PDS training. Thus, we did not engage in extensive hyperparameter tuning. We tried different batch sizes and learning rates, and we did not observe significant qualitative difference. We chose to report the experiments obtained for a standard batch size of 64 and a learning rate of 0.01 for 2-layer MLP and of 0.05 for 4-layer MLP.

G.2. Additional Experiments

We complement the main experiments with two additional figures. Figure 3 uses the same PDS training distribution as Figure 1 but on a sparse (rather than dense) parity, and again shows clear gains from PDS. Figure 4 studies juntas under the same PDS as Figure 2, now with a 4-layer network. Interestingly, in the left plot and for no-PDS, some seeds learn faster with label noise than without; nonetheless, across seeds and noise levels, PDS consistently yields faster and more reliable learning even with the deeper architecture.

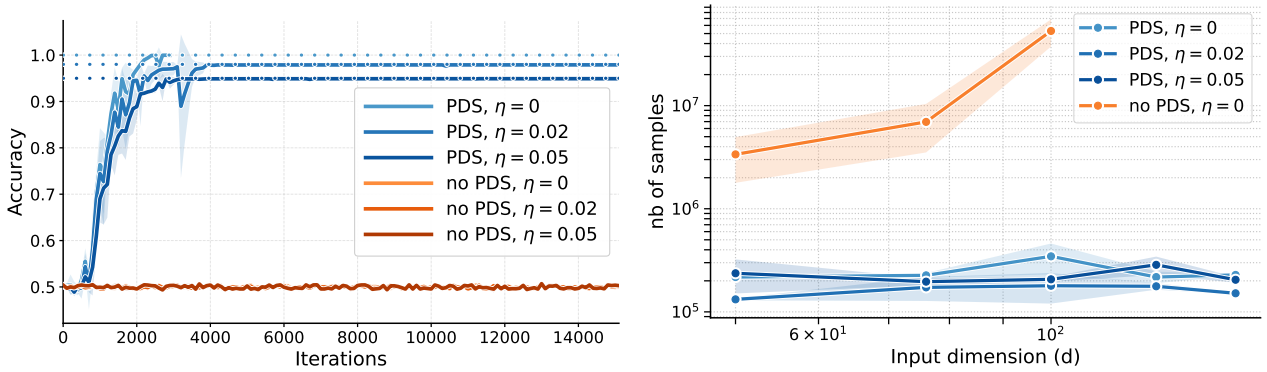


Figure 3. Sparse parity with noise. We compare PDS and standard (no-PDS) learning for a 5-parity with label noise $\eta \in \{0, 0.02, 0.05\}$. (Left) For $d = 50$, we plot the test accuracy on $\mathcal{D}$ versus gradient descent steps for a 4-layer ReLU network trained with SGD (batch size $b = 64$) on fresh samples from $\mathcal{D}'$ (PDS) and from $\mathcal{D}$ (no PDS). Dotted lines show Bayes accuracy (i.e., $1 - \eta$). (Right) We plot the sample complexity to reach within 0.01 of Bayes error versus input dimension. We report the simulations that converged within 10^6 training steps. In both figures, we see that PDS training is markedly more efficient.

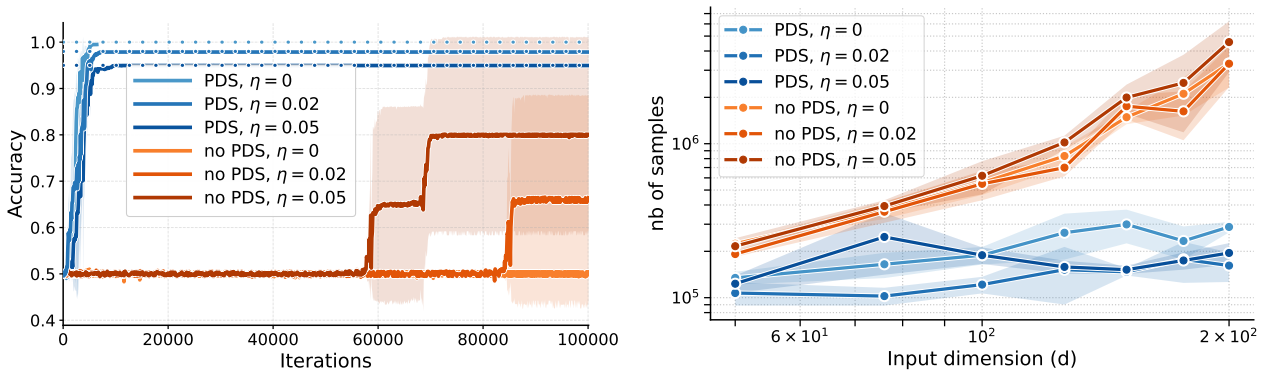


Figure 4. Sparse juntas with noise. We compare PDS and standard (no-PDS) learning for f_9 (see (4)) with label noise $\eta \in \{0, 0.02, 0.05\}$. (Left) For $d = 50$, we plot the test accuracy on $\mathcal{D}$ versus gradient descent steps for a 4-layer ReLU network trained with SGD (batch size $b = 64$) on fresh samples from $\mathcal{D}'$ (PDS) and from $\mathcal{D}$ (no PDS). Dotted lines show Bayes accuracy (i.e., $1 - \eta$). (Right) For f_7 , we plot the sample complexity to reach within 0.01 of Bayes error versus input dimension. In both figures, we see that PDS training is markedly more efficient.